

Dataset Construction for Training LLM to Learn Analog Circuit Knowledge

ZIHAO CHEN*, Fudan University, China

JI ZHUANG*, Fudan University, China

JINYI SHEN, Fudan University, China

XIAOYUE KE, Fudan University, China

XINYI YANG, Fudan University, China

MINGJIE ZHOU, Fudan University, China

ZHUOYAO DU, Fudan University, China

XU YAN, Fudan University, China

ZHOUYANG WU, Fudan University, China

ZHENYU XU, Fudan University, China

JIANGLI HUANG, Fudan University, China

LI SHANG, Fudan University, China

XUAN ZENG, Fudan University, China

FAN YANG[†], Fudan University, China

This paper constructs a textual dataset for training large language models (LLMs) to learn analog circuit knowledge and customizes LLM training techniques. For dataset construction, high-quality textbooks are collected and decomposed into fine-grained learning nodes, which are then used to construct structured question-thinking-solution-answer (QTSA) quadruples using a multi-agent framework to capture both final answers and thought processes. The resulting dataset consists of 7.26M tokens of unlabeled data for continual pre-training (CPT) and 112.65M tokens of labeled data for supervised fine-tuning (SFT). We customize the training techniques including initial model selection, training paradigms, regularization techniques, and practical implementation references. Instruct models are identified as suitable training initialization points, an SFT-centric training paradigm is established (finding that CPT provides marginal benefits compared with SFT due to imbalanced data distribution), and SFT with KL divergence regularization can achieve a 2.71 percentage-point improvement over SFT alone. A practical training implementation method is provided for resource-constrained scenarios. Experiments demonstrate that the dataset and training techniques enhance LLMs' analog circuit knowledge. The trained 32B instruct model achieves 84.59% accuracy on the AMSBench-TQA benchmark, showing a 15.67 percentage-point improvement over the initial model. The trained model also shows capability in the operational amplifier design task based on the Atelier framework.

*These authors contributed equally to this paper.

[†]Corresponding author: yangfan@fudan.edu.cn.

Authors' Contact Information: Zihao Chen, Fudan University, Shanghai, China; Ji Zhuang, Fudan University, Shanghai, China; Jinyi Shen, Fudan University, Shanghai, China; Xiaoyue Ke, Fudan University, Shanghai, China; Xinyi Yang, Fudan University, Shanghai, China; Mingjie Zhou, Fudan University, Shanghai, China; Zhuoyao Du, Fudan University, Shanghai, China; Xu Yan, Fudan University, Shanghai, China; Zhouyang Wu, Fudan University, Shanghai, China; Zhenyu Xu, Fudan University, Shanghai, China; Jiangli Huang, Fudan University, Shanghai, China; Li Shang, Fudan University, Shanghai, China; Xuan Zeng, Fudan University, Shanghai, China; Fan Yang, Fudan University, Shanghai, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7309/2025/1-ART1

<https://doi.org/xx.xxxx/xxxxxxx>

CCS Concepts: • **Hardware** → **Electronic design automation**.

Additional Key Words and Phrases: analog circuit, electronic design automation, large language model, dataset

ACM Reference Format:

Zihao Chen, Ji Zhuang, Jinyi Shen, Xiaoyue Ke, Xinyi Yang, Mingjie Zhou, Zhuoyao Du, Xu Yan, Zhouyang Wu, Zhenyu Xu, Jiangli Huang, Li Shang, Xuan Zeng, and Fan Yang. 2025. Dataset Construction for Training LLM to Learn Analog Circuit Knowledge. *ACM Trans. Des. Autom. Electron. Syst.* 1, 1, Article 1 (January 2025), 30 pages. <https://doi.org/xx.xxxx/xxxxxxx>

1 INTRODUCTION

Analog circuit design is a challenging field, primarily due to the knowledge-intensive nature of the domain. This knowledge spans multiple levels: from fundamental device physics and circuit laws, through the analysis and design of basic building blocks such as operational amplifiers (op-amps) and filters, to the architecture of large-scale systems such as phase-locked loops (PLLs) and analog-to-digital converters (ADCs). Mastering this body of knowledge typically requires years of study and hands-on practice, making analog circuit design a knowledge-intensive field.

Recently, large language models (LLMs), such as the GPT series [1, 29, 38], DeepSeek series [33, 54], and Qwen series [8, 89, 90], have demonstrated remarkable capabilities in acquiring and applying vast amounts of human knowledge from textual data. Given that analog circuit design is a knowledge-intensive task, LLMs emerge as promising tools to assist in this domain. Many works have begun exploring LLM-assisted analog circuit design. For example, training LLMs on curated datasets has proven effective for designing particular circuits such as op-amps [19, 82] and passive power converters [42], while multi-agent frameworks equipped with retrieval-augmented generation (RAG) [49] have empowered general-purpose LLMs in various scenarios including topology design and transistor sizing [2, 46, 56, 72, 95].

Building upon the encouraging results of these task-specific approaches, we are motivated to construct a textual dataset covering analog circuit knowledge for training LLMs to learn.

However, **accessible textual training data for analog circuit knowledge is scarce**. Unlike the software community, where open-source platforms like GitHub and Stack Overflow provide abundant data for LLM training, the analog circuit domain has fewer publicly accessible textual resources. Existing efforts in dataset construction, such as the AMSNet series [74, 75, 79] focusing on visual-to-text transformation, and works like [27, 35] focusing on design variable-performance mappings, have made valuable contributions to the field. However, there remains a need for comprehensive knowledge datasets specifically tailored for LLM training.

Moreover, **effectively training LLMs in this domain requires careful customization of training techniques**. Training strategies vary substantially across domains. For example, code generation typically benefits from continual pre-training (CPT) [40] on massive code repositories [28, 69], while research in mathematics [50, 51] and chemistry [96] suggests that supervised fine-tuning (SFT) [59] plays a more critical role compared with CPT. Given the limited prior experience in this domain, systematically exploring training approaches, including initial model selection, training stage emphasis, algorithm customization, and efficient implementation under resource constraints, requires extensive efforts and computational resources.

In this paper, we construct a domain dataset and customize training techniques for LLMs learning analog circuit knowledge. Our work consists of two main parts: **dataset construction** and **training techniques customization**.

For dataset construction: ① We first collect high-quality textbooks as the raw corpus. Following the learning roadmap of this field, we gather 20 textbooks spanning circuit theory, analog circuit fundamentals, analog integrated circuit design, and advanced topics. ② However, raw textbook

content is unstructured and unlabeled, which is usually insufficient for effective LLM training. To address this, a learning node-based data construction method is proposed to decompose the corpus into fine-grained learning nodes and employ a multi-agent framework to extract structured knowledge in a question-thinking-solution-answer (QTSA) quadruple format, capturing both final answers and underlying thought processes. ③ The resulting dataset consists of 7.26M tokens of unlabeled data for CPT and 112.65M tokens of labeled data for SFT.

For training techniques customization: we investigate training strategies through extensive ablation studies and theoretical analysis, addressing four critical aspects: initial model selection, training paradigm emphasis, algorithm customization, and practical implementation under resource constraints. ① For model selection, we compare base, reasoning, and instruct models, identifying instruct models as the most suitable initialization points through both theoretical analysis and empirical validation, achieving over 15 percentage-point performance improvements over the initial model on the benchmark. ② For training paradigm, we investigate the effectiveness of CPT versus SFT-centric approaches. Given the imbalanced data distribution in our dataset, we establish an SFT-centric training paradigm, finding that CPT provides marginal benefits when followed by SFT. ③ For algorithm customization, we explore incorporating KL divergence regularization into SFT to improve knowledge learning outcomes. Experiments show that this approach achieves a 2.71 percentage-point improvement over traditional SFT. ④ For practical implementation, we provide strategies for resource-constrained scenarios using an asymmetric dual-model deployment technique. Through the above investigation, we document our preliminary findings and lessons learned regarding these training aspects, hoping they can provide useful references for researchers working on similar challenges.

In the experiments, a Qwen2.5-32B-Instruct model trained on the proposed dataset achieves 84.59% accuracy on the AMSBench-TQA benchmark [76], showing a 15.67 percentage-point improvement over the initial model, which validates the effectiveness of the constructed dataset and training techniques. The trained model also demonstrates capability in downstream applications by completing the operational amplifier design task in [72].

These contributions are summarized as follows:

- A textual dataset is constructed for training LLMs to learn analog circuit knowledge. The dataset is built from high-quality textbooks through a learning node-based data construction method that extracts structured QTSA quadruples using a multi-agent framework, resulting in 7.26M tokens of CPT data and 112.65M tokens of SFT data.¹
- Training techniques are customized through ablation studies and theoretical analysis, considering four aspects: model selection, training stage emphasis, algorithm customization, and practical implementation under resource constraints. The findings and lessons learned are documented in this paper to provide references for researchers in our community facing similar challenges.
- The dataset and training techniques are validated through training experiments, demonstrating that they can enhance LLMs' analog circuit knowledge, achieving 84.59% accuracy on AMSBench-TQA with a 15.67 percentage-point improvement over the initial model while preserving downstream task execution capabilities in the Atelier op-amp design framework.

The remainder of this paper is organized as follows. Section 2 introduces the background of this work. Section 3 describes the dataset construction process. Section 4 presents the investigation of training techniques. Section 5 presents the experimental results. Section 6 concludes this paper.

¹To comply with the copyright terms of the source materials, the raw textbook files and the CPT corpus are not distributed. Researchers may obtain a copy of the synthesized SFT dataset by contacting the corresponding author via email and signing a non-disclosure agreement, under the strict condition of academic use only and no redistribution.

2 BACKGROUND

2.1 Knowledge-intensive analog circuit design

Analog circuit design is generally regarded as a demanding discipline in electronic engineering [14, 67]. Designers often need to consider voltage, current, frequency, noise, linearity, and power consumption simultaneously, where these factors are coupled and involve subtle trade-offs. As a result, practitioners typically need knowledge that spans fundamental physics and mathematics, device-level modeling, circuit-level analysis, and physical implementation. This knowledge requirement is the main reason why analog circuit design is treated as a knowledge-intensive field.

For example, consider the operational amplifier (op-amp), a representative analog building block. The required knowledge is structured across multiple levels [5, 14, 37, 62, 67, 70, 78]. ① **Fundamental physical laws:** First, designers rely on physical laws such as Kirchhoff's current and voltage laws and Ohm's law, along with common analysis techniques including nodal analysis, mesh analysis, and Thévenin/Norton equivalent transformations. ② **Mathematical grounding:** Fourier and Laplace transforms are also important, as they support frequency-domain reasoning about circuit behavior. ③ **Transistor device physics and modeling:** Building on these basics, designers need to understand transistor device physics and associated compact models, such as the square-law model, small-signal model, and noise model of transistors, which provide an analytical basis for design decisions. ④ **Single-stage amplifier details:** With device-level knowledge in hand, designers study amplifier topologies progressively: starting from elementary single-stage configurations (e.g., common-source, common-drain, and common-gate stages) and then moving to structures such as differential pairs, single-ended outputs, folded-cascode, and telescopic-cascode structures. ⑤ **Multi-stage amplifier design:** Moving to multi-stage amplifier design introduces additional challenges in frequency response and stability analysis, including pole-zero locations, gain-bandwidth trade-offs, and phase margin considerations. ⑥ **Frequency compensation techniques.** To maintain stable operation, designers often use compensation strategies [37, 48] such as Miller compensation, feedforward compensation, and nested feedback compensation, each with its own applicability constraints and design implications. ⑦ **Physical implementation:** Finally, translating the schematic into a physical layout adds another layer of considerations, including matching strategies, parasitic-aware design, and placement and routing techniques.

This example illustrates that analog circuit design involves extensive knowledge, which motivates us to construct a domain-specific knowledge dataset and train an LLM to acquire part of it.

2.2 Related works of LLM for analog circuit design

The application of LLMs to analog circuit design automation has emerged. LLM applications in topology and parameter design, as the primary decision-making stages in design flow, are presented in this section. Meanwhile, LLMs' roles in other implementation and verification tasks, including executive steps like robust netlist generation, layout optimization, and testbench generation, are also presented. Finally, the progress of specialized datasets and benchmarks is summarized, revealing the data-scarcity in textual domain-knowledge learning that this work addresses.

2.2.1 Design task execution. Early LLM-assisted analog design efforts focus on netlist generation. Works such as [46, 47, 83] formulate netlist construction as Python code synthesis with PySpice. By combining multi-agent coordination and in-context learning, these methods report improved netlist generation accuracy and simulation readiness.

Other works focus on topology and parameter design. These works guide the LLM to conduct pre-defined topology and parameter design processes with existing design recipes to improve the interpretability and design quality, either through dataset construction [16, 19] or agentic retrieval from knowledge bases [56, 72, 95]. LLM-assisted parameter optimization is recently extensively

studied. References [42, 77, 85, 92] enhance existing black-box methods with LLM guidance, while references [2, 20, 57, 58, 94] build sizing workflows driven by the reasoning of agents. Recent works [30, 82] explore combining reinforcement learning with LLMs to explore new topologies.

LLMs are also applied to more downstream tasks. For verification, reference [18] automatically construct executable testbenches for given designs. For physical implementation, reference [55] builds a multi-agent framework to operate layout tools and supports placement optimization.

2.2.2 Datasets and benchmarks. Applying LLMs to analog circuit design requires high-quality training data, which remains scarce. Prior works have produced several datasets and benchmarks, covering various directions: schematic understanding, layout generation, and numerical performance mapping from design variables.

Several efforts focus on visual-to-textual modality transformation. In the AMSnet dataset series [75, 79], schematic images are paired with machine-readable netlists and functional annotations. AMSnet 2.0 [74] further introduces an image segmentation method to recover component positions, improving the schematic parsing robustness and scaling the number of visual-to-textual data samples. On the other hand, the AMSbench [76] benchmark not only contains subsets to test the multimodal LLMs' (MLLMs) capabilities of translating visual circuit information into text, but also includes a text-only question answering subset (AMSbench-TQA) for testing the analog design knowledge without visual input and thereby providing a testing ground for this work. The CircuitSense benchmark [3] evaluates more complex symbolic reasoning capabilities of MLLMs in circuit understanding from schematic images, revealing a critical gap between visual schematic parsing and further symbolic reasoning.

For layout generation, OSIRIS [21] is one of the few available resources that directly target LLM-based layout generation from schematic inputs. It provides a pipeline that generates clean, verified circuit variations with layout, parasitic-extracted performance metrics, and design parameters.

Other works [27, 35] focus on learning topology-to-performance mapping for other types of neural networks, not tailored for LLMs. The OCB dataset [27] offers op-amp instances with circuit topologies and performance labels. It is designed for graph neural networks to learn topology-to-performance mapping or prediction [35]. No natural language annotations are included.

On the other hand, the dataset proposed in this paper does not target a specific design task or step. Instead, it is designed to help LLMs learn theoretical knowledge of analog circuits. In this knowledge-intensive domain, such understanding can help downstream tasks. Besides, this work also investigates the training techniques for LLMs learning analog circuit knowledge. Therefore, this work is different from existing resources, and together they enrich the data spectrum for LLM-assisted analog circuit design.

2.3 Continual Pre-training of LLMs

LLMs that have been trained on generic web-scale corpus can be further adapted by a self-supervised phase on unlabeled text. This procedure, called continual pre-training (CPT) [40], specializes the model while retaining its general abilities. When the continued corpus is domain-specific, CPT can be referred to as domain-adaptive pre-training (DAPT) [34].

Let $\mathcal{D}_{\text{CPT}} = \{x^{(i)}\}_{i=1}^{N_{\text{CPT}}}$ be the continued corpus, where each sequence $x^{(i)} = (x_1^{(i)}, \dots, x_{N_i}^{(i)})$ contains N_i tokens produced by the same tokenizer as the base model, thus ensuring vocabulary compatibility. For a decoder-only architecture such as Qwen series [8, 89, 90], we minimize the auto-regressive negative log-likelihood loss function:

$$\mathcal{L}_{\text{CPT}}(\theta) = - \sum_{i=1}^{N_{\text{CPT}}} \sum_{k=1}^{N_i} \log p_{\theta}(x_k^{(i)} \mid x_{<k}^{(i)}), \quad (1)$$

where θ denotes the trainable parameters in this model, $x_{<k}^{(i)}$ is the left context $(x_1^{(i)}, \dots, x_{k-1}^{(i)})$, and $P_\theta(\cdot|\cdot)$ is the conditional distribution over the vocabulary obtained via a softmax layer. Minimizing this cross-entropy objective continues the original pre-training regime without injecting task-specific bias.

2.4 Supervised Fine-tuning of LLMs

Supervised fine-tuning (SFT) [59] adapts a pre-trained model to the reasoning style required by specific downstream tasks using labeled examples (nowadays typically formatted as question-answer pairs).

Let $\mathcal{D}_{\text{SFT}} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N_{\text{SFT}}}$ be the labeled training set, where $x^{(i)} = (x_1^{(i)}, \dots, x_{N_i}^{(i)})$ denotes the i -th input sequence (question) with N_i tokens, and $y^{(i)} = (y_1^{(i)}, \dots, y_{L_i}^{(i)})$ denotes the corresponding target sequence (answer) with L_i tokens. A decoder-only language model with parameters θ is optimized by the following cross-entropy loss function:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{i=1}^{N_{\text{SFT}}} \sum_{k=1}^{L_i} \log p_\theta(y_k^{(i)} | x^{(i)}, y_{<k}^{(i)}), \quad (2)$$

where θ denotes the trainable parameters in this model, $y_{<k}^{(i)}$ is the left context $(y_1^{(i)}, \dots, y_{k-1}^{(i)})$, and the inner term maximizes the probability of the k -th ground truth token conditioned on the complete input and all previously generated target tokens. Minimizing Eq. (2) aligns the model's output logits with labeled values.

2.5 Categorization of LLMs from Training Aspects

From a training perspective, LLMs fall into three tiers:

(i) Base models. LLMs are first trained from random initialization with a purely auto-regressive objective like Eq. (1) in a corpus with billions to even trillions of tokens [29, 81, 90]. This pre-training imparts broad statistical knowledge, yet leaves the models mostly unaligned with human instructions, so they cannot reliably follow instructions or produce safe and truthful dialogue [1, 29]. Therefore, additional SFT and RL from human feedback (RLHF) [9] stages are typically required before final deployment.

(ii) Instruct models. SFT and RLHF are important and expensive stages in converting a pre-trained base model into a reliable conversational agent, i.e., an instruct model. This process depends on large, diverse, and meticulously curated instruction and preference data pairs. For example, the Qwen2.5 series employs millions of SFT and RLHF data, trains for two epochs with sequences up to 32k tokens, and requires multi-week runs on thousands of GPUs [90]. Although reference [98] shows that roughly ten thousand carefully curated instruction-response pairs can align a model for everyday dialogue, such small-scale supervision still falls short of the performance attained by official models, especially on more demanding tasks.

(iii) Reasoning models. Reasoning models are built by adding RL on top of instruct models, typically initialized with SFT and then refined with GRPO [33] or recent DAPO [93]. This pipeline enhances math and coding performance but requires millions of preference pairs and extensive computational resources. However, the parameter distribution of reasoning models is usually sparsely redundant and highly sensitive, potentially making them prone to collapse or performance degradation during further fine-tuning [10, 63]. Whether reasoning models are suitable for specific domain adaptation depends on various factors and requires careful investigation.

2.6 Typical LLM Domain Customization Pipeline

Conventional workflow typically begins with CPT to incorporate domain text, followed by SFT on instruction data. Due to its higher resource demands, RLHF is sometimes excluded in academic research. The effectiveness of these stages varies significantly across different domains.

In programming tasks, for example, CPT is typically indispensable. Modern code models are usually exposed to billions to trillions of raw code tokens [28, 69]. Such large-scale pre-training yields substantial gains over models that rely on SFT alone, suggesting that language-agnostic syntax and cross-language identifier semantics should be absorbed during CPT.

In contrast, mathematics appears far less reliant on CPT. For example, reference [51] solely trains a 13B model on 5M question-answer pairs, and achieves good performance matching or outperforming code-pretrained models of similar size. Follow-up work [50] further shows that scaling SFT alone enables even a vanilla 7B model to close most of the remaining gap. These findings suggest that general-purpose models already encode latent mathematical structure, which can be effectively unlocked through sufficiently rich SFT.

These mixed outcomes suggest that the training strategy is highly domain-dependent. The optimal balance between CPT and SFT, as well as the relative importance of each stage, depends on various factors including the domain characteristics, data availability, and the scale and distribution of available training data.

2.7 KL Divergence Regularization in LLM Training

LLMs are fundamentally generative models that output probability distributions over tokens. When adapting a pre-trained model to new domains, the model's output distribution may shift significantly during fine-tuning. To control this shift and maintain stability during adaptation, regularization techniques can be employed to constrain the model's output distribution to remain close to the original model. When considering regularization for LLMs, KL divergence naturally emerges as a principled choice, as it directly measures the difference between probability distributions, i.e., the native output space of LLMs. The Kullback–Leibler (KL) divergence [44] quantifies the information loss when approximating one distribution with another:

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}. \quad (3)$$

However, the effectiveness of KL divergence regularization in LLM training is highly context-dependent. For example, in RL for improving LLMs' reasoning capabilities, while GRPO [33] successfully employs KL divergence regularization term for model training, DAPO [93] removes this term and also achieves competitive results. On the other hand, in SFT for LLMs' domain adaptation, KL divergence regularization is less used and the effectiveness is not well studied for the analog circuit domain. Consequently, whether KL divergence regularization benefits domain adaptation tasks, particularly depending on the specific characteristics of the domain and training conditions, remains an open empirical question that warrants careful investigation.

3 Dataset Construction

This section presents the process of constructing the analog circuit textual knowledge dataset for LLM training, which consists of two main stages: Section 3.1 describes the collection and preprocessing of corpora following the learning roadmap of analog circuits; Section 3.2 presents the learning node-based data construction pipeline, which consists of two components: corpus decomposition into learning nodes (Section 3.2.1) and a multi-agent framework for data construction (Section 3.2.2). Section 3.3 provides dataset statistics.

3.1 Corpus Collection and Preprocessing

As discussed in Section 1, accessible textual training data for analog circuit knowledge is scarce, which has become a challenge for applying LLMs in analog circuit design. This section first collects textbooks as the raw corpus, following the learning roadmap of analog circuits.

The theoretical learning of analog circuits is usually divided into four stages: circuit theory, analog circuit basis, analog integrated circuits, and advanced circuit topics. ① **Circuit theory** encompasses basic circuit laws, network analysis methods, time and frequency domain analysis, and other fundamental knowledge, establishing the mathematical foundation for future learning. ② **Analog circuit basis** focuses on device characteristics, basic amplifier circuits, feedback theory, and stability analysis, enabling learners to develop a fundamental understanding of circuit functionality and design techniques. ③ **Analog integrated circuits** delves into the design of core modules such as op-amps, comparators, and reference circuits, as well as the implementation of circuits under CMOS technology. ④ **Advanced circuit topics** covers specialized circuits from op-amps to PLLs.

Table 1. Details of Collected Analog Circuit Corpus

Part 1: Books Categorized by Learning Stage			
Learning stage	Circuit scope		Book number
Circuit theory	Basic passive components and circuits		1
Analog circuit basis	Basic amplifiers and active filters		3
Analog integrated circuits	Various circuits with various scales		5
Advanced circuit topics	Focused circuits for specific applications		11

Part 2: Books Categorized by Circuit Types with Overlap			
Circuit type	Subtype number	Subtype example(s)	Book number
Basic circuits	-	-	4
Amplifiers	6	Op-amp, power amplifier	9
Arithmetic circuits	4	Multiplier, divider	5
Filters	2	Active filter	6
Power management circuits	2	DC-DC, AC-DC	2
Analog-to-Digital converters	8	SAR ADC, Σ - Δ ADC	6
Digital-to-Analog converters	2	Δ - Σ DAC	6
Reference circuits	3	Voltage reference	5
Regulators	5	Linear regulator	5
Transceivers	4	Transmitter, modulator	5
Oscillators	3	VCO, LC oscillator	5
PLLs	1	Charge-pump PLL	6

As shown in Table 1 Part 1, for the first three stages, we select several textbooks with complete content systems, totaling 9 volumes. For the advanced circuit design stage, we select 11 reference books to cover core design techniques in the major application areas. Therefore, we construct a corpus containing 20 core textbooks legitimately for only academic use, which together span at least 12 major circuit types with overlaps, as detailed in Table 1 Part 2, from simple to complex.

At the implementation level, the commercial parser Mathpix is employed to convert the original PDF files into a clean Markdown-formatted corpus, including text recognition, structured processing

of mathematical formulas and tables. Subsequently, a combination of automated scripts and manual adjustments is used to complete the desensitization and denoising of the corpus, finally obtaining an unannotated corpus containing 7.26M tokens.

It should be noted that this method does not incorporate visual modalities in the corpus. On the one hand, this strategy aligns with the development of LLMs in other specialized domains [13, 34]. For example, early-stage LLMs in medicine [31], law [15], and mathematics [7] also relied primarily on domain-specific textual knowledge injection and gained performance improvements on textual benchmark tasks, without incorporating visual modalities. On the other hand, in the analog circuit domain, circuit images contain numerous fine-grained and specialized visual details. The processing techniques for schematic images remain under development [3, 4, 12, 17, 74, 75, 79]. Moreover, visual information in analog circuit textbooks extends far beyond schematics to include photographs, function waveforms, device structure diagrams, and more. A stable, general-purpose image-to-text mapping for the full variety of analog circuit visuals remains an open challenge. Developing such methods is beyond the scope of this paper but is an important direction for future research. Experimental results will demonstrate the effectiveness of this data construction approach.

3.2 Learning Node-based Data Construction

Textbook corpus is unstructured and unlabeled, which is insufficient for LLM training. A learning node-based analog circuit data construction method is proposed, which consists of two steps: decomposing the corpus into fine-grained learning nodes (Section 3.2.1) and employing a multi-agent framework to construct labelled data pairs (Section 3.2.2).

3.2.1 Corpus Decomposition into Learning Nodes. The cleaned corpus remains extremely long text files, making it difficult to effectively generate SFT data from it. A single book could cover different learning stages, encompassing dozens of circuits with vastly different principles and design considerations. Therefore, we need to decompose the corpus into finer-grained units that are both manageable in size and semantically coherent for SFT data construction.

The collected analog circuit textbooks typically feature a hierarchical structure with two-level subsections. As illustrated in Fig. 1, a textbook can be decomposed at multiple granularities: from the entire book, to sections (chapters), and to subsections (learning nodes). For example, a textbook related to circuit theory can usually be divided into many sections, from *basic circuit components* to *frequency response of RLC filters*, each section being further subdivided into several subsections. Based on this observation, we define each subsection as a learning node, i.e., the finest granularity unit for SFT data construction. This is motivated by the fact that subsections represent semantically complete knowledge units: they are neither too fine-grained (which would fragment related concepts) nor too coarse (which would mix different topics), thereby keeping knowledge integrity while being suitable for focused SFT data construction.

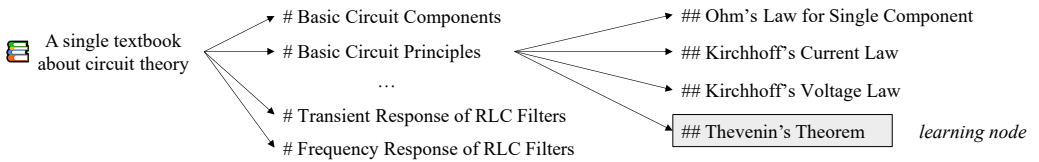


Fig. 1. An example of the granular decomposition of the cleaned corpus. From left to right, the granularity becomes progressively finer. For each book, the first stage is the section, the second stage is the subsection, i.e. the defined learning nodes.

The statistics on the collected corpus primarily validate this decomposition strategy. Out of all identified subsections, we filter out brief, general introductory content and identify 2.69k valid subsections as learning nodes. We subsequently analyze the length of each learning node, finding an average of about 2k tokens per node, which is a relatively moderate length suitable for data construction, neither too short to lack context nor too long to be unwieldy for the subsequent agentic data construction process.

In the implementation, we partition the cleaned corpus according to the extracted chapter and hierarchical information. The Markdown corpus preserves the hierarchical chapter structure with multi-level headings (e.g., # for sections and ## for subsections as shown in Fig. 1). Automated scripts parse these hierarchical heading structures and split the corpus into learning nodes.

3.2.2 Multi-agent Framework for Data Construction. Structured data pairs are constructed from each learning node using LLM agents. Note that knowledge within a learning node may involve not only memorization and comprehension but also reasoning or problem-solving. Therefore, a multi-agent framework consisting of three reasoning agents (Ψ_Q , Ψ_A , and Ψ_P) is employed to construct SFT data pairs. As illustrated in Fig. 2, the framework operates in two main stages: a data generation stage (upper half) where questions are generated and answers are produced, and a post-processing stage (lower half) where the generated data is cleaned and refined.

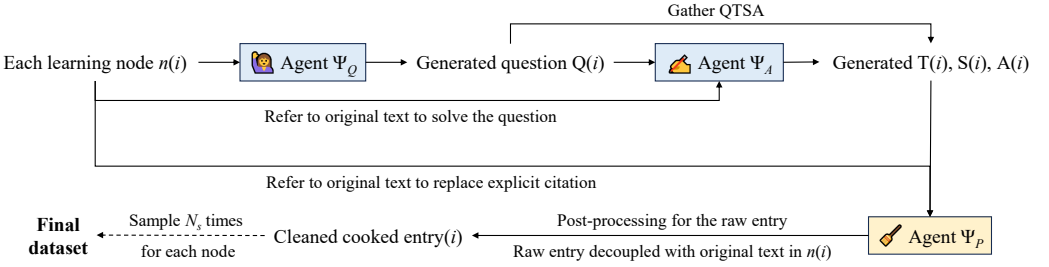


Fig. 2. The multi-agent framework for data construction. The upper half is the data generation stage, while the lower half is the post-processing stage.

To generate data pairs containing both the final answers and the thinking processes, a *question-thinking-solution-answer* (QTSA) quadruple is adopted as the data format. For each learning node $n(i)$, the data entry is defined as:

$$\text{entry}(i) = [Q(i), T(i), S(i), A(i)], \quad (4)$$

where $Q(i)$ represents the generated question, $T(i)$ is the thinking process, which includes informal analysis of the question as well as the entire process of formulating the final solution and answer, $S(i)$ is the refined and relatively structured solving process, and $A(i)$ is the final answer. For training, each data entry $\text{entry}(i)$ is converted into an $(x^{(i)}, y^{(i)})$ pair following the standard SFT format (as defined in Section 2.6), where the input $x^{(i)} = Q(i)$ and the output $y^{(i)}$ consists of $T(i)$, $S(i)$, and $A(i)$ concatenated together, with XML separators (such as $\langle \text{think} \rangle \dots \langle / \text{think} \rangle$ and $\langle \text{answer} \rangle \dots \langle / \text{answer} \rangle$) used to delimit the different components within $y^{(i)}$.

As shown in Fig. 3, the data generation stage involves two agents working sequentially. First, for each learning node $n(i)$, agent Ψ_Q generates a question based on the original content of the learning node:

$$Q(i) = \Psi_Q[n(i)]. \quad (5)$$

Learning Node Transient Response of a Parallel RLC Circuit ...When a resistor R, an inductor L, and a capacitor C are connected in parallel, the natural response of the capacitor voltage $v(t)$ is governed by $\frac{d^2v}{dt^2} + \frac{1}{RC} \frac{dv}{dt} + \frac{1}{LC} v = 0. \quad (1)$ It is convenient to define the damping factor and the undamped natural frequency as $\beta = \frac{1}{RC}, \quad \omega_o = \frac{1}{\sqrt{LC}}. \quad (2)$ Depending on the relative values of β and ω_o, the circuit may be overdamped, critically damped, or underdamped. In the underdamped case, the capacitor voltage can be written as $v(t) = V_o e^{-\beta t/2} \left[\cos(\omega t) - \frac{\beta}{2\omega} \sin(\omega t) \right], \quad \omega = \sqrt{\omega_o^2 - \beta^2/4}. \quad (3)$ The factor $V_o e^{-\beta t/2}$ is the amplitude envelope, showing exponential decay of the oscillation amplitude...
Agent Ψ_Q: Question Construction <think> The node offers several possible anchors, such as damping regimes, the meaning of the envelope, and the relation between exponential decay and a characteristic time. A suitable training question should remain grounded in the source node, require lightweight reasoning, and be easy to normalize later. The envelope in Eq. (3), together with the $1/e \approx 37\%$ observation and the definition of β in Eq. (2), supports a concise multiple-choice question with a clear derivation path. ... </think> Raw Question: According to Eq. (3) and the definition in Eq. (2), in the underdamped parallel RLC circuit with initial capacitor voltage $V_o = 1$ V at $t = 0$, the time at which the amplitude envelope falls to about 37% of its initial value is A) RC B) $1/(RC)$ C) $2/\omega_o$ D) $2RC$
Agent Ψ_A: Answer Generation Raw Thinking: <think> ...We need the time for the oscillation amplitude to decay to 37% of its initial value. According to Eq. (3), the underdamped response is $v(t) = V_o e^{-\beta t/2} [\cos(\omega t) - \frac{\beta}{2\omega} \sin(\omega t)]$. From Eq. (3), the envelope is $V_o e^{-\beta t/2}$. Since $37\% \approx 1/e$, we set $e^{-\beta t/2} = e^{-1}$. Thus, $-\beta t/2 = -1 \Rightarrow t = \frac{2}{\beta}$. Using Eq. (2), $\beta = \frac{1}{RC}$, we get $t = 2RC$. Therefore, the correct option is D... </think> Raw Solution: According to Eq. (3), the voltage response is $v(t) = V_o e^{-\beta t/2} [\cos(\omega t) - \frac{\beta}{2\omega} \sin(\omega t)]$, so the envelope is determined by the exponential factor. Since the text states that $37\% \approx 1/e$, we solve $e^{-\beta t/2} = e^{-1}$, which gives $t = \frac{2}{\beta}$. Substituting Eq. (2), $\beta = \frac{1}{RC}$, yields $t = 2RC$. Raw Answer: <answer>D</answer>
Agent Ψ_P: Post-processing <think> The raw QTS is correct but still depends on the source-node context through references such as “According to Eq. (3)” and “Using Eq. (2).” To make the sample self-contained, these references should be replaced by the explicit content they point to. In particular, Eq. (3) should be unfolded into the underdamped response and its envelope $V_o e^{-\beta t/2}$, while Eq. (2) should be rewritten directly as $\beta = 1/(RC)$. The reasoning chain and final answer remain unchanged; only context-dependent references are removed. ... </think> Post-processed Question (cooked Q): In an underdamped parallel RLC circuit, the capacitor voltage can be written as $v(t) = V_o e^{-\beta t/2} [\cos(\omega t) - \frac{\beta}{2\omega} \sin(\omega t)]$, where $\beta = \frac{1}{RC}$. If $V_o = 1$ V at $t = 0$, the time at which the amplitude envelope falls to about 37% of its initial value is A) RC B) $1/(RC)$ C) $2/\omega_o$ D) $2RC$ Post-processed Thinking (cooked T): <think> We need the time for the oscillation amplitude to decay to 37% of its initial value. The underdamped capacitor voltage is $v(t) = V_o e^{-\beta t/2} [\cos(\omega t) - \frac{\beta}{2\omega} \sin(\omega t)]$, so the amplitude envelope is $V_o e^{-\beta t/2}$. Because $37\% \approx 1/e$, we set $e^{-\beta t/2} = e^{-1}$. This gives $-\beta t/2 = -1 \Rightarrow t = \frac{2}{\beta}$. Using $\beta = \frac{1}{RC}$, we obtain $t = 2RC$. Therefore, the correct option is D. </think> Post-processed Solution (cooked S): The underdamped response is $v(t) = V_o e^{-\beta t/2} [\cos(\omega t) - \frac{\beta}{2\omega} \sin(\omega t)]$, so its amplitude envelope is $V_o e^{-\beta t/2}$. To find the time at which the amplitude decays to about 37% of its initial value, use $37\% \approx \frac{1}{e}$, and solve $e^{-\beta t/2} = e^{-1}$. Hence, $t = \frac{2}{\beta}$. Since $\beta = \frac{1}{RC}$, the result is $t = 2RC$. Post-processed Answer (cooked A): <answer>D</answer>

Fig. 3. An illustrative example of the QTS data construction process. Some content is abbreviated for brevity.

The prompt design for Ψ_Q requires it to propose a guided question that reflects the information in $n(i)$ based on the original text content. Specifically, the question must be grounded in the original text content and should be answerable using the original content with reasoning and calculations. This design ensures that the generated questions are aligned with the learning node content. Fig. 3 shows an example where agent Ψ_Q proposes a question based on the RLC circuit transient response knowledge.

Subsequently, agent Ψ_A produces the answer based on its own knowledge and abilities, the original learning node content $n(i)$ and the generated question $Q(i)$:

$$\text{entry}(i) = \Psi_A[n(i), Q(i)]. \quad (6)$$

Specifically, DeepSeek-R1 [33], a reasoning model that naturally generates both thinking processes and formal answers, is employed as the agent in this data generation framework. The model automatically generates `<think>...</think>` tags containing the reasoning process, which we extract as $T(i)$. The content outside the `<think>` tags constitutes the formal answer, which we extract as $S(i)$. Furthermore, the final answer should be wrapped in `<answer>...</answer>` tags, allowing the content within these tags to be extracted as $A(i)$ from $S(i)$. Fig. 3 shows an example where agent Ψ_A generates the raw thinking, solving process, and final answer. To ensure comprehensive knowledge coverage in each learning node, we independently sample each learning node N_S times.

The generated data undergoes further postprocessing and cleaning through agent Ψ_P , whose functionality can be seen in Fig. 3. First, the agent removes ill-formatted items, including those that do not conform to the QTSA structure or contain obvious errors. Second, it identifies referential expressions in the answers, such as references to formulas, figures, or tables from the original corpus, and replaces them with explicit content. For example, when the generated answer contains phrases like "as shown in Eq. (3)" or "according to the definition in Eq. (2)" (highlighted with underlines in Fig. 3), the post-processing agent Ψ_P locates the corresponding formula in the original learning node and rewrites the reference as the actual content. This decoupling process helps ensure that each sample remains relatively self-contained even when removed from the original book layout, thereby improving the usability of the dataset.

3.3 Dataset Statistics

Table 2 summarizes the dataset scale after the automated construction and screening steps. The original corpus was decomposed into 2.69k unlabeled CPT data, containing 7.26M tokens. Using the multi-agent methods introduced in this section, each learning node was independently sampled repeatedly. After filtering out duplicate samples and obviously erroneous samples, we obtained 15.31k data entries. Fig. 3 presents an example of the QTSA data entry. Compared with other domains, the scale of this dataset remains limited. Nevertheless, under the experimental settings of this paper, the constructed dataset can serve as the input for LLM training.

For data correctness verification, we manually checked 500 randomly sampled instances from the SFT dataset. Of these, 455 entries (91.00%) were found to be correct and well-formed. The remaining 45 entries, including 2 entries with formatting issues and 43 entries with factual errors or hallucinations. The performance improvement in subsequent experiments on the AMSBench-TQA test benchmark can serve as evidence of the dataset's effectiveness.

For original textbook content and SFT dataset similarity filtering, the cosine similarity between each QTSA sample and its corresponding source learning node text is computed using the sentence embedding model from [68]. Samples with a similarity score exceeding 0.8 are considered as potential contaminated samples, which are further checked manually. In total, 236 SFT samples exhibited a similarity score exceeding 0.8 (none of them exceeded 0.9). We manually reviewed all 236 entries against the original textbook learning nodes to assess whether they contained near-verbatim reproduction of original textbook content. Finally, 140 samples are identified that retained phrasing too close to the original textbook content. These small amount of samples are removed from the final SFT dataset.

Table 2. Details of the Constructed Dataset

Training stage	Data entry number	Data token number
CPT	2.69k	7.26M
SFT	15.31k	112.65M

4 Training Techniques Customization

This section presents the training techniques for effectively utilizing the constructed dataset to train an LLM in learning analog circuit knowledge. Through extensive ablation studies, we investigate training strategies for analog circuit domain adaptation, addressing four aspects that are critical for effective LLM training: Section 4.1 investigates the selection of appropriate pre-trained models as initialization points, identifying instruct models as the better choice over base and reasoning models; Section 4.2 establishes an SFT-centric training paradigm, demonstrating that CPT provides only marginal benefits; Section 4.3 investigates the effectiveness of KL divergence regularization in SFT; and Section 4.4 provides practical implementation strategies for resource-constrained scenarios.

4.1 Model Selection for Training Initialization

The selection of an appropriate pre-trained model as the initialization point is crucial for training an LLM in learning analog circuit knowledge. As discussed in Section 2.5, the current LLM landscape comprises three categories, each of which presents unique characteristics that impact their suitability for analog circuit knowledge learning.

(i) Base models: impractical and unnecessary. While base models offer theoretical flexibility for domain customization, we identify that adopting a base model is both impractical and unnecessary in our scenario. On the one hand, high-quality instruct checkpoints like Qwen2.5-32B-Instruct are already publicly available, leaving expensive retraining from a base model unnecessary. On the other hand, reproducing training from base models is infeasible, since it requires the proprietary SFT and RLHF datasets used in official post-training, which are not publicly released for leading open-source models [8, 89, 90]. Therefore, we further investigate whether a reasoning model or an instruct model is more suitable for training an analog circuit LLM.

(ii) Reasoning models: superior but fragile. While reasoning models excel at complex tasks, they are sensitive to further training on domain-specific data. For example, a reasoning model such as DeepSeek-R1 [33] is trained through GRPO-like RL methods [33, 93], where the extensive online sampling with reward feedback customizes a model parameter distribution for difficult reasoning tasks [36, 87]. References [41, 88] further demonstrate that RL processes applied to LLMs exhibit brittleness, i.e., small parameter changes can lead to large, unpredictable shifts in output. This indicates that heavy RL training narrows the model's parameter numerical distribution, reducing its capacity to adapt to further domain-specific fine-tuning [88].

To assess whether such fragility arises in the context of analog circuit design, the following ablation study is conducted. As shown in Table 3, fine-tuning QwQ-32B [90], an open-source reasoning model through SFT, results in a performance degradation from 80.78% to less than 75% on the benchmark. This degradation validates our hypothesis and is consistent with prior observations [41, 60, 88] regarding the fragility of RL-optimized model parameter distributions.

(iii) Instruct models: aligned and adaptable. On the other hand, instruct models provide a more suitable initialization for training LLMs in the analog circuit domain. An instruct model, such as Qwen2.5-32B-Instruct [90], is derived from a base model through extensive SFT followed by relatively lightweight RLHF. The RLHF stage of instruct models often achieves instruction following and safety alignment [63], and its impact on the model's parameter distribution is limited [64] when compared to the GRPO-like RL process to train reasoning models [86]. This difference in RL intensity accounts for the different behaviors of reasoning and instruct models during further domain-specific fine-tuning. Recent studies also compare the different roles of SFT and RL. References [25, 65] indicate that SFT retains the model's ability to learn new tasks, while reasoning models become brittle under further domain-specific fine-tuning because heavy RL training overly tightens the model's parameter distribution and exhausts its plasticity [41, 60, 88].

As shown in Section 5, fine-tuning Qwen2.5-32B-Instruct can achieve a performance improvement of more than 15 percentage points, validating that analog circuit knowledge can be effectively integrated through the further training of instruct models.

4.2 SFT-centric Training Paradigm

While Section 2.6 establishes that training strategies are highly domain-dependent, the optimal configuration for analog circuits remains unexplored and requires investigation. Analog circuit knowledge is both complex and domain-specific, yet our dataset exhibits a significant imbalance: the corpus contains only 7.26M tokens for CPT versus 112.65M tokens for SFT (approximately 16× larger). Prior work suggests that CPT requires substantial data volumes typically exceeding 100M tokens [34], which is well above our CPT corpus size. Given these characteristics, we hypothesize that: CPT could provide minimal benefits due to insufficient data scale, while SFT alone may achieve comparable performance by directly teaching domain knowledge through instruction tuning.

To validate this hypothesis, we conduct a controlled ablation study in Section 5, comparing SFT-only versus CPT+SFT pipelines. Our results confirm that the CPT+SFT pipeline yields only a marginal 0.17 percentage-point improvement over SFT alone (see Table 4), which could be negligible and likely within the range of random variation. Therefore, an SFT-centric approach is more efficient for training an analog circuit LLM.

4.3 SFT with KL Divergence Regularization

4.3.1 The Motivation of Using KL Divergence Regularization. Given our domain-specific SFT dataset is costly yet limited, maximizing the utility of this dataset becomes important. Analog circuit knowledge is both complex, requiring the model to learn domain-specific knowledge while preserving general capabilities of understanding and instruction following.

However, fine-tuning an LLM on new data could cause the model to forget some of its previously learned knowledge [52]. When we inject new analog circuit knowledge into the model (noted as ΔK_{inject}), the model also loses some of its prior knowledge (noted as ΔK_{forget}). For conceptual illustration (not a formal measurement), the total domain knowledge gain ΔK_{gain} can be roughly represented as:

$$\Delta K_{\text{gain}} = \Delta K_{\text{inject}} - \Delta K_{\text{forget}}. \quad (7)$$

In this work, the analog circuit training dataset is relatively small and difficult. Under such conditions, the forgetting term ΔK_{forget} can become quite large and harm overall performance. Therefore, any technique that reduces forgetting should help increase the total knowledge gain.

A KL divergence regularization term during SFT can help reduce this forgetting. At first glance, this may seem counterproductive: SFT encourages the model to fit new data with the cross-entropy loss, while KL term penalizes deviation from the original model. However, the two terms work in a complementary way. SFT provides the necessary gradient signal to inject domain knowledge, while the KL term acts as a mild regularizer that prevents the model from drifting too far from its pre-trained behavior. This combination is widely used in RL [33, 63, 71], where KL penalties can constrain policy updates and maintain training stability [43, 97]. Recent work [99] also shows that KL-regularized SFT serves as an anchoring mechanism, mitigating over-memorization and parameter distribution drift in SFT. In our setting, the KL term serves the same purpose: it limits overfitting to the small analog circuit dataset and helps preserve the original general capabilities. The effect of this combination is a controlled adaptation process achieving better knowledge gain.

4.3.2 Mathematical Formulation and Validation. To implement this approach, we adopt the following loss function:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(y_{\text{predict}}, y_{\text{label}}) + \lambda \cdot D_{\text{KL}}(p_{\text{predict}} \parallel p_{\text{ref}}), \quad (8)$$

where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss in Eq. (2), measuring the discrepancy between the prediction y_{predict} and the label y_{label} ; λ is the weighting coefficient balancing the two objectives; and $D_{\text{KL}}(p_{\text{predict}} \parallel p_{\text{ref}})$ applies the KL divergence definition in Eq. (3) to our SFT training scenario.

Specifically, we denote the trainable parameters of the current model and the reference model as θ and θ_0 , respectively. The KL divergence definition in Eq. (3) is instantiated in our SFT training scenario as the following more detailed form:

$$D_{\text{KL}}(p_{\theta} \parallel p_{\theta_0}) = \sum_{v \in \mathcal{V}} p_{\theta}(v \mid x^{(k)}) \log \left[\frac{p_{\theta}(v \mid x^{(k)})}{p_{\theta_0}(v \mid x^{(k)})} \right], \quad (9)$$

where $v \in \mathcal{V}$ represents all possible tokens in the dictionary, $p_{\theta}(v \mid x^{(k)})$ is the output probability assigned by the current model to token v given input $x^{(k)}$, and $p_{\theta_0}(v \mid x^{(k)})$ is the output probability assigned by the reference model to the same token. This formulation compute the distribution difference at each token position between the current model and the reference model.

Empirical validation in Section 5 shows that this KL divergence regularization approach can achieve a 2.71 percentage-point improvement over traditional SFT for a 32B model, and for smaller 7B models, the improvement is even larger.

4.4 Practical Implementation for Resource-constrained Scenarios

4.4.1 The Challenge of Implementing SFT with KL Regularization. In the EDA community, computational resources for LLM training are often limited. Therefore, beyond the algorithmic effectiveness, hardware feasibility of KL-regularized SFT should also be considered. Unlike standard SFT, KL-regularized SFT requires maintaining two LLMs during training: a target model (abbreviated as Tar. in Fig. 4) to be optimized, and a frozen reference model (abbreviated as Ref. in Fig. 4) that provides the baseline distribution for the KL term. This means that an additional forward pass of the reference model must be executed, and the outputs of both models must be retained simultaneously for KL divergence computation. This leads to the following challenges:

(i) Without distributed training, GPU memory would overflow. Consider our experimental setup: training a 32B target model at BF16 precision. The target model alone occupies 64GB for parameters. Keeping both target and reference models fully resident on each GPU doubles the requirement to 128GB. Adding gradients, optimizer states, activations, and communication buffers pushes the per-GPU memory demand beyond the 141GB limit of an advanced NVIDIA H200 GPU.

(ii) Existing distributed SFT techniques are not readily applicable to dual-model scenarios. Mainstream distributed SFT framework DeepSpeed ZeRO-3 [66] registers hooks on each module to gather partitioned parameters before forward pass and release them after backward pass. This scheduling logic assumes a single model and a fixed execution order: forward, then backward. However, two models are used in this paper: a target model that is being trained, and a frozen reference model that only runs forward passes. The reference model triggers forward hooks but never enters backward pass, leaving gathered parameters in an unreleased state. Meanwhile, the interleaved execution of the two models sends conflicting requests to the same parameter state machine. ZeRO-3 cannot resolve these conflicts because its hook system was not designed for more than one model. Actually, naive attempts to run both models under ZeRO-3 resulted in frequent hangs or out-of-memory errors. It seems hard to directly use a ZeRO-3-like distributed training framework for this dual-model setup without modifications to its core C++ code.

4.4.2 The Customized Implementation Scheme. An asymmetric dual-model deployment scheme is proposed to address the above challenges, enabling SFT with KL regularization. Only the trainable target model is optimized using DeepSpeed ZeRO-3 sharding, while the frozen reference model is loaded in full on each GPU as a static local copy. Fig. 4 illustrates the overall approach. The

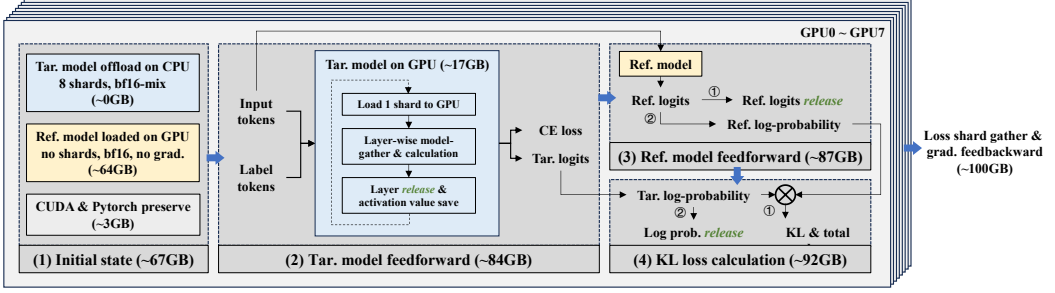


Fig. 4. The implementation of SFT with KL divergence regularization. This diagram specifically demonstrates the data flow during the forward propagation and gradient backpropagation process, along with the estimates of the memory usage per device for each step. The peak memory usage is tested to be 101.37GB per device.

following description uses a setup example: 8 GPUs, a 32B model, BF16 precision, and a maximum sequence length of 8192 tokens.

(i) Asymmetric dual-model deployment. The scheme applies DeepSpeed ZeRO-3 only to the target model. ZeRO-3 shards its parameters, gradients, and optimizer states across GPUs and offloads inactive parameters to CPU when possible, reducing per-GPU memory usage. In contrast, the reference model is frozen and only participates in forward inference for KL computation. It requires neither gradients nor optimizer states, so it is loaded entirely on each GPU without ZeRO-3 management. This asymmetric deployment introduces extra static memory cost but avoids the scheduling conflicts that arise when both models run under ZeRO-3. It enables data-parallel training (at least one sample per GPU) while keeping the implementation simple and stable.

(ii) Target model forward pass. At the start of training, each GPU already holds the full reference model (64GB) with CUDA runtime overhead, resulting in an initial memory footprint of about 67GB. The target model then performs its forward pass under ZeRO-3. As shown in Fig. 4, parameters are gathered layer by layer and released immediately after use, so the full set of training parameters never resides persistently on a single GPU.

(iii) Cross-entropy loss computation. After the target model forward pass, the cross-entropy loss for SFT is computed. Intermediate activations that are not needed for subsequent KL computation are explicitly deleted to free memory. Only the target model output logits required for the KL computation are retained.

(iv) Reference model forward pass. Once the target logits are obtained, the reference model performs its forward pass independently on each GPU. Because the reference model is frozen, no computation graph is built for backpropagation. During this stage, per-GPU memory usage rises to approximately 87GB. Loading the reference model locally sacrifices some memory but simplifies the execution path and improves training stability.

(v) KL divergence computation. With both target and reference model outputs available, the KL divergence is computed. At this point, logits from both models briefly coexist in memory, creating a temporary peak. For numerical stability, log-softmax is applied to each set of logits and the KL divergence is computed in log space. Immediately after computation, logits and temporary tensors are explicitly deleted. At this stage, per-GPU memory usage exceeds 90GB.

(vi) Target model backward pass and parameter update. As defined in Eq. (8), the total loss consists of the supervised cross-entropy term and the KL divergence term. Backpropagation and parameter updates are performed only on the target model. During this phase, gradient computation and ZeRO-3 parameter reconstruction cause memory usage to rise again. In practice, due to CUDA

caching and framework-level preallocation, the per-GPU memory footprint stabilizes at 101.37GB, confirming that the scheme runs stably on a standard 8×H200 server.

4.4.3 Other details and applicability. This scheme is realized through targeted customization at the Python training pipeline level. First, at initialization, only the target model is handed over to the DeepSpeed engine; the reference model is loaded separately as a frozen static model on each GPU, bypassing ZeRO-3 scheduling. Second, the loss computation flow is rewritten: target model outputs are obtained first to compute the supervised loss, then reference model outputs are computed under `torch.no_grad()`, and finally the KL divergence is calculated online and combined into the total loss. Third, to suppress transient memory spikes during long-sequence dual-model training, fine-grained tensor cleanup is inserted at stage boundaries, explicitly deleting intermediate variables that are no longer needed.

This scheme does not rely on modifying DeepSpeed’s bottom C++/CUDA kernels. Instead, it builds upon existing training frameworks to enable stable and reproducible dual-model training with KL divergence regularization under resource-constrained conditions. For EDA researchers working on LLM applications, this scheme shares a feasible solution to train as large as 32B-parameter models with KL regularization on standard 8-GPU servers.

5 EXPERIMENTS

5.1 Basic Environment

5.1.1 Hardware Environment. We utilize a server with 8 NVIDIA H200 SXM GPUs, each equipped with 141GB memory (700W), providing sufficient computational resources for medium-scale model training. For circuit simulation, we use an Intel Xeon Platinum 8358 CPU.

5.1.2 Software Environment. Our software infrastructure is built on CUDA 12.2, ensuring compatibility with the Transformers framework [84] and DeepSpeed framework [66]. All models are loaded using BF16 precision for memory efficiency, with flash-attention techniques [22, 23] disabled to maintain numerical stability during KL-divergence calculations. For inference and evaluation, we leverage vLLM [45] for high-throughput parallel processing.

5.2 Training Settings

5.2.1 Model Choice. The training begins with Qwen 2.5-32B-Instruct, a medium-sized model with good baseline performance [90]. Ablation studies also use the equally sized reasoning model QwQ-32B [80] and the Qwen 2.5-7B-Instruct model [90] for KL weight experiments.

5.2.2 General-Domain Data Mixing. During the SFT phase, we incorporate the OpenThoughts dataset [32] with 20k samples alongside our analog-circuit domain data, maintaining a balanced scale between general and domain-specific samples, which is a common practice in domain-specific model training.

5.2.3 Hyper-parameter Settings. We use a cosine-annealing scheduler with a 10% warm-up fraction, empirically determining that a maximum learning rate of 2×10^{-6} works well for both CPT and SFT, with modest variations showing negligible impact on performance. Our training configuration allocates 1 sequence per GPU batch with 8-step gradient accumulation, resulting in a global batch size of 64 that ensures training stability. We set the maximum sequence length at 8,192 tokens and implement packing for shorter sequences to maximize computational efficiency. The weighting coefficient λ for KL divergence regularization is set to 0.1 according to the empirical training results on small-scale models.

5.3 Evaluation Settings

5.3.1 AMSBench-TQA Benchmarking. Trained models are evaluated on AMSBench-TQA, a third-party benchmark in [76]. AMSBench-TQA is a human-crafted, text-only benchmark specifically designed to assess analog circuit knowledge through textual question-answering, which aligns with our research objective of evaluating pure textual knowledge understanding in analog circuits. While AMSBench encompasses multiple subsets, the other subsets primarily focus on vision-language capabilities, which evaluate different competencies than our current research objectives. AMSBench-TQA comprises a fixed set of multiple-choice problems, making automatic grading straightforward. All trained models share a system prompt and a text generation temperature value of 0. To ensure fair evaluation, data leakage between the SFT dataset and the benchmark is examined. Using the embedding model from [68], cosine similarities between the SFT dataset and the benchmark samples are calculated. Pairs with a similarity above 0.75 were manually checked to confirm whether any benchmark samples or their near variants were present in the SFT dataset. 76 contaminated benchmark samples were identified and removed, yielding a deduplicated test set with 1181 samples in our experiment.

5.3.2 Atelier Op-amp Design Task. Additionally, the Atelier framework [72] is employed to validate if the model can assist op-amp design. Atelier is a multi-agent framework that combines LLM agents, a fixed workflow, and knowledge bases for circuit design. This framework requires the agent to perform topology selection, topology modification, and circuit analysis. Topology selection and modification refer to the agent choosing or adjusting the circuit structure based on design specs and simulation outcomes; circuit analysis requires the agent to refine the topology into a HSPICE netlist and specify the tunable parameters along with ranges. The design specification (spec) in the experiment is gain > 85 dB, gain-bandwidth product (GBW) > 5 MHz, phase margin (PM) > 55 degrees, power < 250 μ W, and the comprehensive figure of merit (FoM) is expected to be large. The maximum number of iterations is set to 4.

5.4 Benchmarking Result Analysis

Table 3 presents the performance of various baseline models and our model on this benchmark. The trained models are compared with commercial LLMs including Claude-Sonnet-4 [6], DeepSeek-R1 [33], DeepSeek-V3 [54] and GPT-4o [38]. The open-source baseline model Qwen2.5-32B-Instruct [90], the reasoning model QwQ-32B [80] with a comparable scale, and the recent Qwen3-32B model [89] are included for comparison.

Table 3. Performance Comparison between Our Model and Baseline Models on AMSBench-TQA Benchmark

Starting point	Model scale	Method	Group	Accuracy (%)
Claude-Sonnet-4	Unknown	—	Baseline	85.77
DeepSeek-R1	671B	—	Baseline	84.93
Qwen2.5-32B-Instruct	32B	SFT+KL	Experiment	84.59
DeepSeek-V3	671B	—	Baseline	83.66
Qwen3-32B	32B	—	Baseline	80.86
QwQ-32B	32B	—	Baseline	80.78
GPT-4o	Unknown	—	Baseline	73.24
Qwen2.5-32B-Instruct	32B	—	Baseline	68.92

5.4.1 Comparison with Similar-Scale Baseline Models. As shown in Table 3, the model trained from Qwen2.5-32B-Instruct with KL-regularized SFT achieves an accuracy of 84.59%, marking a 15.67 percentage-point improvement over the baseline instruct model Qwen2.5-32B-Instruct (68.92%). This improvement demonstrates the effectiveness of the proposed dataset and training method.

Besides, compared to Qwen2.5-32B-Instruct, the more advanced models from the same series, QwQ-32B and Qwen3-32B, achieve accuracies of 80.78% and 80.86%, respectively, and also outperform Qwen2.5-32B-Instruct (68.92%).

QwQ-32B, which builds on the Qwen2.5-32B-Instruct baseline, improves its reasoning ability through large-scale RL [80]. Although not specifically trained on analog circuit data, its strong reasoning capability allows it to derive correct answers through logical deduction, partly compensating for the lack of domain knowledge. For example, as shown in Fig. 5, when analyzing noise in a classic CMOS op-amp, QwQ-32B applies basic formulas and correctly reasons that increasing g_{m1} reduces the first-stage noise contribution, while lowering the g_{m3}/g_{m1} ratio increases first-stage gain and thus lowers input-referred noise.

Qwen3-32B, as a new-generation model, not only adopts a more advanced architecture, but also adds trillions of tokens during the pre-training stage, with a particular increase in the STEM (science, technology, engineering, and mathematics) data [89]. This targeted training optimization enhances its overall ability, explaining why Qwen3-32B achieves competitive performance despite lacking specialized training on analog circuit data.

5.4.2 Comparison with Larger-Scale Commercial Models. As shown in Table 3, larger-scale baseline models, Claude-Sonnet-4, DeepSeek-R1 and DeepSeek-V3 achieve 85.77%, 84.93% and 83.66% accuracy, respectively. The performance of our model is comparable to that of these larger-scale commercial models on this benchmark.

Especially, Claude-Sonnet-4, released later than DeepSeek-R1, is a newer model that generally outperforms DeepSeek-R1 in multiple evaluations [6]. Therefore, its top rank in Table 3 is expected. On the other hand, GPT-4o's relatively ordinary performance (73.24%) reflects its divergent training objectives, as GPT-4o is designed as a model for daily interaction [38] rather than for professional technical assistance.

DeepSeek-V3 achieves 83.66% accuracy. This performance is primarily attributed to two factors. First, DeepSeek-V3 adopts a Mixture-of-Experts architecture with 671B total parameters, 20.97 times larger than our 32B model [54]. Second, the model is pre-trained on trillions of diverse and high-quality tokens, including substantial amounts of challenging math and code data [54]. As the model used in our knowledge construction pipeline, DeepSeek-R1 further achieves 84.93% accuracy, slightly higher than DeepSeek-V3 and our model. This improvement stems from the RL training applied to the 671B DeepSeek-V3, which enhances the model's reasoning capabilities through GRPO [33]. Although Claude-Sonnet-4 achieves a slightly higher accuracy (85.77%), DeepSeek-R1 still delivers competitive performance on the benchmark. Given our resource and budget limitations, DeepSeek-R1 is selected as the backbone LLM for data generation because of its reasonable balance between performance and cost.

5.4.3 Ablation Study I: Analysis of CPT. The CPT phase employs unsupervised domain texts for training, to enhance the model's understanding of analog circuit domain language patterns. As shown in Table 4, Qwen2.5-32B-Instruct (CPT) achieves 70.96% accuracy, demonstrating that CPT alone yields a 2.04 percentage-point performance improvement for the original Qwen2.5-32B-Instruct. However, after subsequent SFT, the benefits of CPT become negligible. For example, Qwen2.5-32B-Instruct (CPT+SFT) achieves 82.05% accuracy, only 0.17 percentage points higher than Qwen2.5-32B-Instruct (SFT). Moreover, compared to Qwen2.5-32B-Instruct (SFT+KL), Qwen2.5-32B-Instruct (CPT+SFT+KL) underperforms by 0.51 percentage points.

Table 4. Performance Comparison for Ablation Study I on AMSBench-TQA Benchmark

Starting point	Model scale	Method	Group	Accuracy (%)
Qwen2.5-32B-Instruct	32B	SFT+KL	Experiment	84.59
Qwen2.5-32B-Instruct	32B	CPT+SFT+KL	Ablation I	84.08
Qwen2.5-32B-Instruct	32B	CPT+SFT	Ablation I	82.05
Qwen2.5-32B-Instruct	32B	SFT	Ablation I	81.88
Qwen2.5-32B-Instruct	32B	CPT	Ablation I	70.96
Qwen2.5-32B-Instruct	32B	—	Baseline	68.92

This phenomenon can be explained from two perspectives. From a data scale perspective, as shown in Table 2, the CPT dataset is only 1/16 the size of the SFT dataset, resulting in a relatively limited impact on final model parameters. From a parameter update magnitude perspective, the supervised learning signal in the SFT phase is stronger [40, 59], producing parameter updates far exceeding those from CPT, thereby overriding the subtle adjustments introduced by CPT in our scenario with imbalanced CPT and SFT data distribution.

5.4.4 Ablation Study II. Analysis of KL Regularization. To demonstrate the performance improvement brought by the KL divergence regularization term in our experiments, repeated experiments were conducted using three different random seeds. In addition to the default seed 42, denoted as Seed A in this ablation study, we randomly selected seeds 123 and 9345, denoted as Seed B and Seed C, respectively, to perform comparative ablation experiments between SFT alone and SFT+KL. As shown in Table 5, the KL divergence regularization term consistently improves performance over SFT across all three seeds, with an average improvement of 2.46 percentage points. Therefore, the KL divergence regularization term does improve the performance of SFT under this paper’s analog circuit knowledge learning scenario.

Fig. 5 provides a detailed example of this phenomenon. While Qwen2.5-32B-Instruct (SFT+KL) correctly analyzes CMOS op-amp noise and selects the correct option, the SFT-only model, Qwen2.5-32B-Instruct (SFT), makes a critical error: it claims the g_{m3}/g_{m1} ratio only affects gain, not input-referred noise. This is fundamentally incorrect. In multi-stage op-amps, the input-referred noise includes contributions from all stages, with later-stage noise referred to the input by dividing by the preceding gain. For a classical two-stage op-amp where the first-stage gain is approximately g_{m1}/g_{m3} , the total input-referred noise becomes: $V_{n,in}^2 = V_{n1}^2 + V_{n3}^2 \cdot (g_{m3}/g_{m1})^2$, where V_{n1}^2 and V_{n3}^2 are the

Table 5. Performance Comparison for Ablation Study II on AMSBench-TQA Benchmark

Starting point	Model scale	Method	Group	Accuracy (%)
Qwen2.5-32B-Instruct	32B	SFT+KL	Seed A	84.59
Qwen2.5-32B-Instruct	32B	SFT+KL	Seed B	84.17
Qwen2.5-32B-Instruct	32B	SFT+KL	Seed C	82.98
Qwen2.5-32B-Instruct	32B	SFT	Seed A	81.88
Qwen2.5-32B-Instruct	32B	SFT	Seed B	81.46
Qwen2.5-32B-Instruct	32B	SFT	Seed C	81.03
Qwen2.5-32B-Instruct	32B	—	Baseline	68.92

Question What are the advantages of fully differential sample-and-hold circuits for high-speed usage? (A) A fully differential sample-and-hold circuit offers lower power and improved noise immunity, making it suitable for high-speed applications. (B) The fully differential sample-and-hold circuit provides higher precision and performance in high-speed applications due to its balanced architecture. (C) This type of circuit reduces signal distortion and enhances speed by minimizing common-mode noise, which is critical for high-speed operations. (D) The circuit's design allows for faster signal processing and improved accuracy, which are essential for high-speed applications.
Qwen2.5-32B-Instruct (SFT+KL) Response ✓ To determine the correct answer, we need to analyze the advantages of a fully differential sample-and-hold circuit for high-speed applications. Let's break down each option: Option A claims that a fully differential sample-and-hold circuit offers lower power consumption and improved noise immunity. While improved noise immunity is a valid advantage, lower power consumption is not typically highlighted as a primary benefit for high-speed applications. The focus is more on performance and accuracy. Option B states that the circuit provides higher precision and better performance due to its balanced architecture. This is accurate because a fully differential circuit inherently reduces common-mode noise and improves precision, which are critical for high-speed applications. Option C mentions that the circuit reduces signal distortion and enhances speed by minimizing common-mode noise. While minimizing common-mode noise is a benefit, the enhancement of speed is not directly attributed to this feature. The primary advantage is the reduction of distortion and improvement in precision. Option D claims that the circuit allows for faster signal processing and improved accuracy. While improved accuracy is a benefit, the emphasis on faster signal processing is not as strong as the precision and performance aspects highlighted in other options. After evaluating all options, Option B is the most accurate as it directly addresses the balanced architecture and its impact on precision and performance ...
QwQ-32B Response ✗ Okay, so I need to figure out the correct answer for the question about the advantages of a fully differential sample-and-hold circuit in high-speed applications. Let me start by recalling what a sample-and-hold circuit does. It's used to capture an analog signal at a specific time and hold that voltage level for a certain period, right... First, I remember that differential circuits, like differential amplifiers, have some benefits over single-ended ones. They can reject common-mode noise because they process the difference between two signals. That's probably important for noise immunity. High-speed applications often deal with noise issues, so that's a key point. Looking at the options: Option A says lower power and improved noise immunity. Hmm, I'm not sure about the power part. Maybe differential circuits can have some power benefits, but I'm not certain. The noise immunity part makes sense because of common-mode rejection. Option B mentions higher precision and better performance due to balanced architecture. Balanced architecture does help in reducing distortion and noise, which contributes to precision. That sounds plausible. Option C talks about reducing signal distortion and enhancing speed by minimizing common-mode noise. Minimizing common-mode noise would reduce distortion, and maybe that allows for faster operation since the signal is cleaner. But does that directly enhance speed? Option D says faster signal processing and improved accuracy. Faster processing might come from the design, but I need to think if differential circuits inherently allow for higher speed... Wait, the question is about advantages specifically for high-speed. High-speed applications need both speed and accuracy. Let me think again ... <i>(...The continual thinking process becomes increasingly circular and indecisive...)</i> Wait, the question is about advantages for high-speed. The key advantages of a fully differential S/H circuit would be: better common-mode noise rejection, leading to lower distortion and higher accuracy, and the balanced architecture can lead to faster settling times, allowing for higher sampling rates. So the advantages include both reduced distortion and enhanced speed. Option C says exactly that... <i>(...Enter a loop, repeatedly reconsidering options B and C...)</i>
Qwen2.5-32B-Instruct (SFT) Response ✗ To determine the advantages of a fully differential sample-and-hold circuit for high-speed applications, we need to analyze each option and evaluate the key benefits mentioned. Option A: A fully differential sample-and-hold circuit offers lower power consumption and improved noise immunity, making it suitable for high-speed applications. While improved noise immunity is a valid advantage, lower power consumption is not typically highlighted as a primary benefit for high-speed applications. High-speed circuits often require more power to achieve faster processing speeds. Therefore... Option B: The fully differential sample-and-hold circuit provides higher precision and better performance in high-speed applications due to its balanced architecture. A balanced architecture in a fully differential circuit can indeed lead to higher precision and better performance. The balanced nature helps in canceling out common-mode noise and reducing distortion, which are critical for high-speed applications. This option seems accurate. Option C: This type of circuit reduces signal distortion and enhances speed by minimizing common-mode noise, which is critical for high-speed operations. Reducing signal distortion and minimizing common-mode noise are significant advantages of fully differential circuits. These features directly contribute to better performance in high-speed applications. This option is also accurate. Option D: The circuit's design allows for faster signal processing and improved accuracy, which are essential for high-speed analog applications. Faster signal processing and improved accuracy are indeed essential for high-speed analog applications. The balanced architecture of a fully differential circuit supports these requirements. This option is accurate as well. Now, comparing the options: Option A mentions lower power consumption, which is not a primary advantage for high-speed applications. Options B, C, and D all highlight key advantages such as precision, performance, reduced distortion, minimized noise, faster processing, and improved accuracy. These are all critical for high-speed operations. However, the question asks for the advantages specifically for high-speed applications. All three options (B, C, D) are valid, but we need to choose the most comprehensive one. Option D directly states faster signal processing and improved accuracy, which are essential for high-speed analog applications, making it the most precise and relevant...

Fig. 5. An example of models' response to a question in the AMSBench-TQA benchmark. Note that for the same question, besides the model trained with SFT+KL and the original QwQ-32B, DeepSeek-V3, DeepSeek-R1 and Claude-Sonnet-4 also provide the correct answer. Their logs are similar to those shown in this figure, and therefore, are not repeated here.

noise power spectral densities of the first and second stages, g_{m1} and g_{m3} are the transconductances of the input and second-stage transistors. This equation clearly shows that increasing g_{m1}/g_{m3} reduces the second-stage noise contribution. The SFT-only model fails to recognize this noise referral mechanism and gets the relationship exactly backwards.

This benchmark performance improvement brought by KL regularization can be explained from the following perspectives. When fine-tuning for a specific domain via SFT, LLMs tend to overfit the small-scale target domain data, causing excessive parameter updates and leading to the forgetting of previously acquired capabilities such as language understanding, instruction following, and general reasoning [52, 59]. In the RL setting, numerous works [33, 63, 71, 73] have already demonstrated that a KL term can mitigate forgetting and steadily improve model performance.

In SFT, the cross-entropy loss provides the necessary gradient signal to inject domain-specific knowledge, while the KL regularization term acts as a mild constraint that prevents the model's parameter distribution from drifting too far from the reference model. This allows the model to absorb new knowledge while preserving its original capabilities as much as possible. Reference [91] experimentally demonstrates that adding a KL divergence term to the loss function can improve performance on the target domain without sacrificing other capabilities. Reference [99] further reports significant gains across tasks including mathematical reasoning, medical knowledge anchoring, and code generation by introducing a KL regularization term to curb excessive distribution shift during SFT. These works align with the findings of this paper and jointly affirm the effectiveness of KL regularization for domain-specific SFT in analog circuit knowledge learning.

5.4.5 Ablation Study III. KL Regularization Weight Settings. The weight of the KL divergence regularization term is commonly set around 0.1. To determine a suitable value of λ for our experiments effectively, the ablation study III on the 7B model is conducted. Fixing the random seed at 42, λ values ranging from 0 to 0.5 are tested, with a particular focus around 0.1. As shown in Table 6, when λ increases from 0 to 0.1, model performance improves from 55.63% to 79.85%; when λ further increases from 0.1 to 0.5, performance declines from 79.85% to 76.55%. This indicates that setting λ to 0.1 achieves a balance between retaining general capabilities and acquiring new knowledge.

Moreover, for the 7B model, the introduction of KL divergence yields a performance improvement of 24.22 percentage points compared with SFT alone, which is larger than the 2.71 percentage-point gain for the 32B model. This is because smaller LLMs have limited parameter capacity and are more susceptible to losing general capabilities during fine-tuning, as parameter updates tend to overwrite

Table 6. Performance Comparison for Ablation Study III on AMSBench-TQA Benchmark

Starting point	Model scale	Method	λ value	Accuracy (%)
Qwen2.5-7B-Instruct	7B	SFT+KL	0.00	55.63
Qwen2.5-7B-Instruct	7B	SFT+KL	0.01	60.80
Qwen2.5-7B-Instruct	7B	SFT+KL	0.05	77.30
Qwen2.5-7B-Instruct	7B	SFT+KL	0.08	79.42
Qwen2.5-7B-Instruct	7B	SFT+KL	0.09	79.42
Qwen2.5-7B-Instruct	7B	SFT+KL	0.10	79.85
Qwen2.5-7B-Instruct	7B	SFT+KL	0.11	78.41
Qwen2.5-7B-Instruct	7B	SFT+KL	0.12	78.49
Qwen2.5-7B-Instruct	7B	SFT+KL	0.20	77.98
Qwen2.5-7B-Instruct	7B	SFT+KL	0.30	77.82
Qwen2.5-7B-Instruct	7B	SFT+KL	0.40	77.22
Qwen2.5-7B-Instruct	7B	SFT+KL	0.50	76.55

previously acquired knowledge. As noted in references [11, 39], small-scale models possess a very narrow tolerance margin for knowledge retention; unconstrained SFT can cause damage to their general capabilities. In contrast, larger LLMs such as the 32B variant possess adequate parameter redundancy, and can experience less forgetting and overfitting during SFT [11, 39]. Consequently, larger LLMs can tolerate parameter updates induced by SFT to a certain extent without excessively compromising their original capabilities, and the benefit of KL regularization for larger LLMs is correspondingly smaller.

5.4.6 Ablation Study IV: Analysis of Reasoning Models. QwQ-32B is a reasoning model optimized through large-scale RL [80], with its parameter distribution finely tuned to handle complex general reasoning tasks. The ablation study IV shown in Table 7 reveals a consistent phenomenon: any domain-specific fine-tuning applied to this model leads to performance degradation in our scenario. Across experiments, the maximum accuracy drop reaches 7.88 percentage points. SFT leads to substantial performance collapse regardless of whether KL divergence regularization is employed and whether CPT is used before SFT.

Table 7. Performance Comparison for Ablation Study IV on AMSBench-TQA Benchmark

Starting point	Model scale	Method	Group	Accuracy (%)
QwQ-32B	32B	—	Baseline	80.78
QwQ-32B	32B	SFT+KL	Ablation IV	74.51
QwQ-32B	32B	SFT	Ablation IV	74.34
QwQ-32B	32B	CPT+SFT+KL	Ablation IV	73.07
QwQ-32B	32B	CPT+SFT	Ablation IV	72.90

These results indicate that reasoning models exhibit sensitivity to subsequent parameter updates, and injecting additional domain knowledge into such models is difficult. This phenomenon can be attributed to the fragility of parameter distributions induced by RL optimization. Reasoning models like QwQ-32B undergo extensive online sampling and reward feedback during large-scale RL training [33, 80, 93]. Such high-intensity RL training of LLMs can cause the mapping from reward to optimal policy to be discontinuous rather than smooth, driving the model to converge to steep local parameter regions [88], where further parameter perturbations can trigger discontinuous shifts in output behavior, rendering these models resistant to further domain knowledge injection [24, 41, 60].

5.4.7 Ablation Study V: Analysis of General Data and Domain Data. To demonstrate the effectiveness of the data used in this paper, ablation study V was conducted. The results are presented in Table 8.

Training solely with analog circuit domain data yields a substantial performance improvement, achieving an accuracy of 82.81%, which represents a gain of 13.89 percentage points over the baseline. This result indicates that the proposed SFT dataset is independently effective for injecting analog circuit domain knowledge, and does not rely on the capabilities of the general dataset.

General data was included not to inject domain knowledge, but to help the model retain its general reasoning and instruction-following abilities during domain training [26]. Including a mixture of general and domain data is standard practice in domain-specific LLM development [53, 61]. The OpenThoughts dataset [32] contains general reasoning examples across mathematics, coding, and science, and is known to improve reasoning performance. Therefore, Qwen2.5-32B-Instruct, a non-reasoning model, improved to 79.93% after training on OpenThoughts, without injecting domain knowledge. However, this gain remains lower than that achieved with domain data alone, confirming that the domain dataset provides information not captured by general reasoning data.

Mixed training with both general and domain data achieves the highest accuracy (84.59%), which is 1.78 percentage points above the domain-only result. This gain reflects the different roles of the

Table 8. Performance Comparison for Ablation Study V on AMSBench-TQA Benchmark

Starting point	Model scale	Method	Group	Accuracy (%)
Qwen2.5-32B-Instruct	32B	SFT+KL	Mixed Data	84.59
Qwen2.5-32B-Instruct	32B	SFT+KL	Only Domain Data	82.81
Qwen2.5-32B-Instruct	32B	SFT+KL	Only General Data	79.93
Qwen2.5-32B-Instruct	32B	—	Baseline	68.92

two data sources: domain data provides task-specific knowledge, while general data helps maintain the model's general capabilities and reduces overfitting to the domain distribution. References [53, 61] indicate that diversity in training data contributes to improving overall LLM capabilities, particularly in scenarios with imbalanced domain data distributions, where diversity itself serves as an effective signal to prevent overfitting. This is also consistent with our results: combining general and domain data preserves general ability while incorporating domain knowledge, leading to the best overall performance.

In summary, this ablation study demonstrates that the domain dataset constructed in this work plays an independent and more important role in knowledge injection.

5.4.8 Summary. Based on the aforementioned analysis, our experiments lead to the following findings: (i) SFT with KL divergence regularization outperforms traditional SFT in adapting to the domain of analog circuit knowledge. (ii) In the context of analog circuits, the imbalanced data distribution between the CPT and SFT stages makes direct SFT much more effective than CPT. (iii) Model selection is pivotal for achieving training success, with general instruct models proving to be more suitable as starting points for domain adaptation than highly specialized reasoning models.

5.5 Agentic Op-amp Design Task Study

To further evaluate whether the trained models can serve as design assistants, we integrate the LLMs trained from Qwen2.5-32B-Instruct with different configurations into the Atelier op-amp design framework [72].

Experimental results are presented in Table 9. The model trained with a mixture of domain and general data achieves a design success rate of 7/10 and an average figure of merit (FoM) of 533.48, representing a 42.5% improvement over the baseline. Taking the design trajectory illustrated in Fig. 6 and Fig. 7 as an example: in the first iteration, the agent selects the NMCF topology, and the circuit analysis module converts it into a netlist followed by parameter optimization. At this stage, the PM is only 35.88°, indicating insufficient stability. In the second iteration, the agent introduces a nulling resistor to improve PM, and the modified design satisfies all requirements with an FoM of 1107.26. In the subsequent two iterations, the agent continues to explore alternative topological adjustments. These observations indicate that the trained model Qwen2.5-32B-Instruct (SFT+KL) can understand specs, leverage domain knowledge to select appropriate topologies, and make targeted adjustments based on feedback, demonstrating capabilities of a design assistance agent.

Table 9. Atelier Op-amp design Task Results

Experimental group	Success rate	Gain (dB)	GBW (MHz)	PM (°)	Power (μ W)	FoM
Full pipeline	7/10	75.94	7.47	40.39	109.19	533.48
General data only	7/10	66.67	5.45	40.19	91.59	451.96
Domain data only	6/10	57.83	5.41	35.39	91.41	378.06
baseline	7/10	65.40	5.54	42.18	102.49	374.42

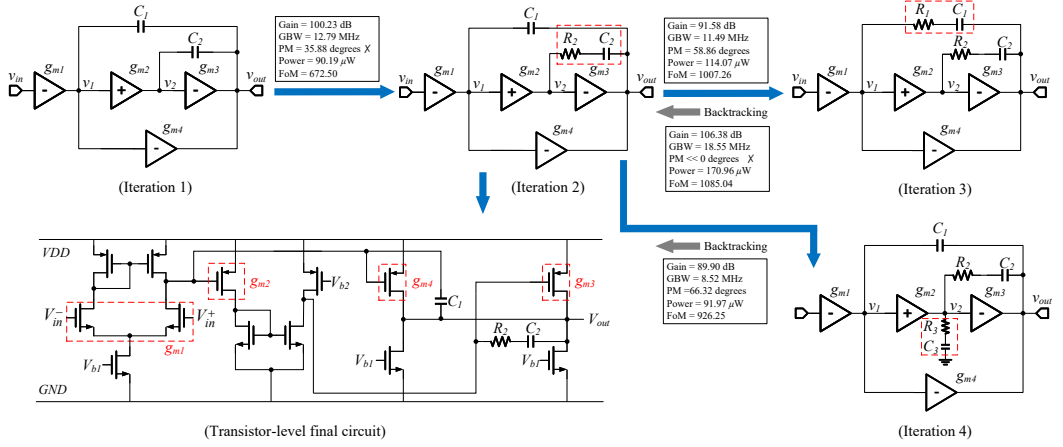


Fig. 6. The op-amp design trajectory of Qwen2.5-32B-Instruct (SFT+KL) under the Atelier framework. For each iteration, the circuit designed by the agent and its corresponding performance metrics are presented.

The model trained only on domain data achieves a success rate of 6/10 and an average FoM of 378.06, which is marginally improved from the baseline while the success rate slightly declines. This trend aligns with the fact that fine-tuning solely on domain data tends to cause degradation of general instruction-following capabilities [53, 61]. In our experiment, failure cases in this group usually involve netlist syntax errors or illegal output formats, suggesting that weaknesses in general abilities most directly affect the agent's performance when executing real tasks.

The model trained exclusively on the OpenThoughts general dataset achieves a success rate of 7/10 and an average FoM of 451.96, a 20.7% improvement over the baseline, but still lower than the domain-general mixed group. OpenThoughts [32] encompasses reasoning tasks across mathematics, programming, and science; its role is not to inject analog circuit expertise but rather to help the model preserve and improve general instruction-following and reasoning capabilities [26]. Consequently, despite lacking domain knowledge, this model can still successfully conduct design tasks by relying on general reasoning and by producing correctly formatted outputs, avoiding frequent instruction-following errors.

The domain-general mixed group attains the highest FoM (533.48) while maintaining the success rate, demonstrating the effective integration of domain knowledge and general capabilities. This result is consistent with prior work [53, 61] indicating that interleaving general data with domain data helps balance knowledge acquisition and capability retention, preventing excessive deviation from the original model distribution and yielding better performance in tasks.

6 CONCLUSION

This paper constructs a textual dataset for LLMs to learn analog circuit knowledge and customizes training approaches. A dataset consisting of 7.26M tokens for CPT and 112.65M tokens for SFT is constructed, where structured QTSA quadruples are produced for the SFT stage to capture domain knowledge. Several findings that may help researchers facing similar challenges are: instruct models serve as suitable initialization points, SFT-centric approaches outperform other pipelines in our setting, and KL divergence regularization can achieve a 2.71 percentage-point improvement over SFT alone. Experiments demonstrate that a 32B instruct model trained on our dataset achieves 84.59% accuracy on AMSBench-TQA, showing a 15.67 percentage-point improvement over the initial model, and demonstrates capability in the downstream op-amp design task.

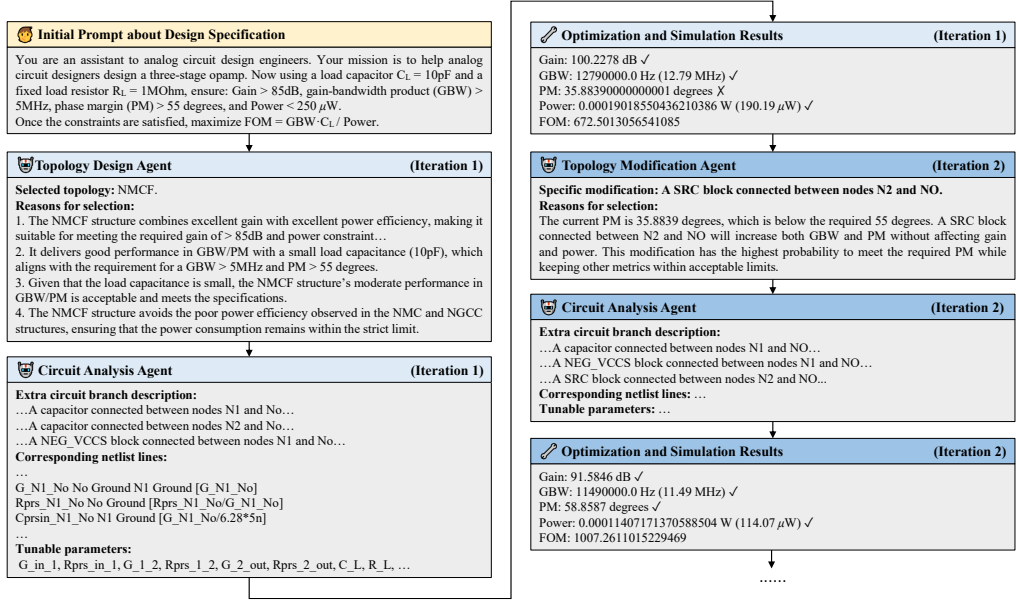


Fig. 7. The chatlog of Qwen2.5-32B-Instruct (SFT+KL) within the Atelier op-amp design framework. The model assumes the roles of two agents: a topology design/modification agent and a circuit analysis agent. The detailed dialogue of the first two iterations is shown here. Subsequent content is omitted, but the overall workflow can still be seen in Fig. 6. Besides, due to space limitations, some details of the content have been omitted using ellipses.

7 Acknowledgments

This work is partially supported by the National Natural Science Foundation of China under research project 92373207, and by the Science and Technology Commission of Shanghai Municipality under grant 25JD1400400.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Mohsen Ahmadzadeh, Kaichang Chen, and Georges Gielen. 2025. AnaFlow: Agentic LLM-based workflow for reasoning-driven explainable and sample-efficient analog circuit sizing. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–7.
- [3] Arman Akbari, Jian Gao, Yifei Zou, Mei Yang, Jinru Duan, Dmitrii Torbunov, Yanzhi Wang, Yihui Ren, and Xuan Zhang. 2026. CircuitSense: A Hierarchical Circuit System Benchmark Bridging Visual Comprehension and Symbolic Reasoning in Engineering Design Process. In *International Conference on Learning Representations (ICLR)*.
- [4] Saoud Aldowaish, Yashwanth Karumanchi, Kai-Chen Chiang, Soroosh Noorzad, and Morteza Fayazi. 2026. SINA: A Circuit Schematic Image-to-Netlist Generator Using Artificial Intelligence. *arXiv preprint arXiv:2601.22114* (2026).
- [5] Charles K Alexander, Matthew NO Sadiku, and Matthew Sadiku. 2007. *Fundamentals of electric circuits*. McGraw-Hill Higher Education Boston, MA, USA.
- [6] Anthropic. 2025. System Card: Claude Opus 4 & Claude Sonnet 4. (May 2025). <https://www-cdn.anthropic.com/4263b940cabb546aa0e3283f35b686f4f3b2ff47.pdf>
- [7] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. 2023. Llemma: An open language model for mathematics. *arXiv preprint*

- arXiv:2310.10631* (2023).
- [8] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
 - [9] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
 - [10] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073* (2022).
 - [11] Joshua Barron and Devin White. 2025. Too Big to Think: Capacity, Memorization, and Generalization in Pre-Trained Transformers. *arXiv preprint arXiv:2506.09099* (2025).
 - [12] Johannes Bayer, Leo van Waveren, and Andreas Dengel. 2024. Modular Graph Extraction for Handwritten Circuit Diagram Images. *arXiv preprint arXiv:2402.11093* (2024).
 - [13] Iz Beltagy, Kyle Lo, and Arman Cohan. 2019. SciBERT: A pretrained language model for scientific text. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3615–3620.
 - [14] T.C. Carusone, D. Johns, and K. Martin. 2011. *Analog Integrated Circuit Design*. Wiley.
 - [15] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. 2020. LEGAL-BERT: The muppets straight out of law school. In *Findings of the association for computational linguistics: EMNLP 2020*. 2898–2904.
 - [16] Chen-Chia Chang, Wan-Hsuan Lin, Yikang Shen, Guanglei Zhou, Yiran Chen, and Xin Zhang. 2026. LaMAGIC: Advanced Circuit Formulations for Language-Model-based Topology Generation for Analog Integrated Circuits. *ACM Transactions on Design Automation of Electronic Systems* (2026).
 - [17] Hong Cai Chen, Longchang Wu, Ming Gao, Lingrui Shen, Jiarui Zhong, and Yipin Xu. 2024. DocEDA: Automated extraction and design of analog circuits from documents with large language model. *arXiv preprint arXiv:2412.05301* (2024).
 - [18] Weiyu Chen, Chengjie Liu, Wenhao Huang, Jinyang Lyu, Mingqian Yang, Yuan Du, Li Du, and Jun Yang. 2025. Analogtester: A large language model-based framework for automatic testbench generation in analog circuit design. In *2025 International Symposium of Electronics Design Automation (ISED)*. IEEE, 201–207.
 - [19] Zihao Chen, Jiangli Huang, Yiting Liu, Fan Yang, Li Shang, Dian Zhou, and Xuan Zeng. 2024. Artisan: Automated Operational Amplifier Design via Domain-specific Large Language Model. In *2024 61th ACM/IEEE Design Automation Conference (DAC)*. 1–6.
 - [20] Zihao Chen, Jiayin Wang, Ziyi Sun, Ji Zhuang, Jinyi Shen, Xiaoyue Ke, Li Shang, Xuan Zeng, and Fan Yang. 2026. White-Box Op-Amp Design via Human-Mimicking Reasoning. *arXiv preprint arXiv:2601.21321* (2026).
 - [21] Giuseppe Chiari, Michele Piccoli, and Davide Zoni. 2026. OSIRIS: Bridging Analog Circuit Design and Machine Learning with Scalable Dataset Generation. In *International Conference on Learning Representations (ICLR)*.
 - [22] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.
 - [23] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [24] Muxi Diao, Lele Yang, Wuxuan Gong, Yutong Zhang, Zhonghao Yan, Yufei Han, Kongming Liang, Weiran Xu, and Zhanyu Ma. 2026. Entropy-Adaptive Fine-Tuning: Resolving Confident Conflicts to Mitigate Forgetting. *arXiv preprint arXiv:2601.02151* (2026).
 - [25] Bowen Ding, Yuhan Chen, Jiayang Lv, Jiyao Yuan, Qi Zhu, Shuangshuang Tian, Dantong Zhu, Futing Wang, Heyuan Deng, Fei Mi, et al. 2025. Rethinking Expert Trajectory Utilization in LLM Post-training. *arXiv preprint arXiv:2512.11470* (2025).
 - [26] Fei Ding and Baiqiao Wang. 2025. Improved supervised fine-tuning for large language models to mitigate catastrophic forgetting. *arXiv preprint arXiv:2506.09428* (2025).
 - [27] Zehao Dong, Weidong Cao, Muhan Zhang, Dacheng Tao, Yixin Chen, and Xuan Zhang. 2023. CktGNN: Circuit graph neural network for electronic design automation. *arXiv preprint arXiv:2308.16406* (2023).
 - [28] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
 - [29] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.
 - [30] Jian Gao, Weimin Fu, Xiaolong Guo, Weidong Cao, and Xuan Zhang. 2025. Eva: An efficient and versatile generative engine for targeted discovery of novel analog circuits. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*.

- IEEE, 1–7.
- [31] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare (HEALTH)* 3, 1 (2021), 1–23.
 - [32] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. 2025. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178* (2025).
 - [33] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
 - [34] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A Smith. 2020. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 8342–8360.
 - [35] Yuxuan Hou, Jianrong Zhang, Hua Chen, Min Zhou, Faxin Yu, Hehe Fan, and Yi Yang. 2024. CktGen: Specification-Conditioned Analog Circuit Generation. *arXiv preprint arXiv:2410.00995* (2024).
 - [36] Maggie Huan, Yuetai Li, Tuney Zheng, Xiaoyu Xu, Seungone Kim, Minxin Du, Radha Poovendran, Graham Neubig, and Xiang Yue. 2025. Does math reasoning improve general llm capabilities? understanding transferability of llm reasoning. *arXiv preprint arXiv:2507.00432* (2025).
 - [37] Johan Huijsing. 2011. *Operational amplifiers*. Springer.
 - [38] Raisa Islam and Owana Marzia Moushi. 2024. Gpt-4o: The cutting-edge advancement in multimodal llm. *Authorea Preprints* (2024).
 - [39] Damjan Kalajdzievski. 2024. Scaling laws for forgetting when fine-tuning large language models. *arXiv preprint arXiv:2401.05605* (2024).
 - [40] Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. 2023. Continual pre-training of language models. *arXiv preprint arXiv:2302.03241* (2023).
 - [41] Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. 2023. Understanding the effects of rlhf on llm generalisation and diversity. *arXiv preprint arXiv:2310.06452* (2023).
 - [42] Dimple Vijay Kochar, Hanrui Wang, Anantha P Chandrakasan, and Xin Zhang. 2025. Ledro: Llm-enhanced design space reduction and optimization for analog circuits. In *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 141–148.
 - [43] Tomasz Korbak, Ethan Perez, and Christopher Buckley. 2022. RL with KL penalties is better viewed as Bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 1083–1091.
 - [44] Solomon Kullback. 1951. Kullback-leibler divergence. *Tech. Rep.* (1951).
 - [45] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
 - [46] Yao Lai, Sungyoung Lee, Guojin Chen, Souradip Poddar, Mengkang Hu, David Z Pan, and Ping Luo. 2025. Analogcoder: Analog circuit design via training-free code generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 379–387.
 - [47] Yao Lai, Souradip Poddar, Sungyoung Lee, Guojin Chen, Mengkang Hu, Bei Yu, Ping Luo, and David Z Pan. 2025. Analogcoder-pro: Unifying analog circuit generation and optimization via multi-modal llms. *arXiv preprint arXiv:2508.02518* (2025).
 - [48] Ka Nang Leung and Philip KT Mok. 2001. Analysis of multistage amplifier-frequency compensation. *IEEE transactions on circuits and systems I: fundamental theory and applications* 48, 9 (2001), 1041–1056.
 - [49] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
 - [50] Chen Li, Weiqi Wang, Jingcheng Hu, Yixuan Wei, Nanning Zheng, Han Hu, Zheng Zhang, and Houwen Peng. 2024. Common 7b language models already possess strong math capabilities. *arXiv preprint arXiv:2403.04706* (2024).
 - [51] Chengpeng Li, Zheng Yuan, Hongyi Yuan, Guanting Dong, Keming Lu, Jiancan Wu, Chuanqi Tan, Xiang Wang, and Chang Zhou. 2023. Mugglemath: Assessing the impact of query and response augmentation on math reasoning. *arXiv preprint arXiv:2310.05506* (2023).
 - [52] Sen Lin, Peizhong Ju, Yingbin Liang, and Ness Shroff. 2023. Theory on forgetting and generalization of continual learning. In *International Conference on Machine Learning*. PMLR, 21078–21100.
 - [53] Zhenqing Ling, Daoyuan Chen, Liuyi Yao, Qianli Shen, Yaliang Li, and Ying Shen. 2025. Diversity as a reward: Fine-tuning llms on a mixture of domain-undetermined data. *arXiv preprint arXiv:2502.04380* (2025).

- [54] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [55] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. LayoutCopilot: an LLM-powered multiagent collaborative framework for interactive analog layout design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 8 (2025), 3126–3139.
- [56] Chengjie Liu, Weiyu Chen, Anlan Peng, Yuan Du, Li Du, and Jun Yang. 2024. Ampagent: An llm-based multi-agent system for multi-stage amplifier schematic design from literature for process and performance porting. *arXiv preprint arXiv:2409.14739* (2024).
- [57] Chang Liu and Danial Chitnis. 2025. Eesizer: Llm-based ai agent for sizing of analog and mixed signal circuit. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2025).
- [58] Chang Liu, Emmanuel A Olowe, and Danial Chitnis. 2025. LLM-based AI agent for sizing of analog and mixed signal circuit. In *2025 23rd IEEE Interregional NEWCAS Conference (NEWCAS)*. IEEE, 90–94.
- [59] Ben Mann, N Ryder, M Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1 (2020).
- [60] Kohsei Matsutani, Shota Takashiro, Gouki Minegishi, Takeshi Kojima, Yusuke Iwasawa, and Yutaka Matsuo. 2025. RL squeezes, sft expands: A comparative study of reasoning llms. *arXiv preprint arXiv:2509.21128* (2025).
- [61] Chenlin Ming, Chendi Qu, Mengzhang Cai, Qizhi Pei, Zhuoshi Pan, Yu Li, Xiaoming Duan, Lijun Wu, and Conghui He. 2025. Ideal: Data equilibrium adaptation for multi-capability language model alignment. *arXiv preprint arXiv:2505.12762* (2025).
- [62] James William Nilsson and Susan A Riedel. 2015. *Electric circuits*. Pearson Upper Saddle River, NJ.
- [63] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.
- [64] Samarth Raina, Saksham Aggarwal, Aman Chadha, Vinija Jain, and Amitava Das. 2025. D-STEER-Preference Alignment Techniques Learn to Behave, not to Believe—Beneath the Surface, DPO as Steering Vector Perturbation in Activation Space. *arXiv preprint arXiv:2512.11838* (2025).
- [65] Neel Rajani, Aryo Pradipta Gema, Seraphina Goldfarb-Tarrant, and Ivan Titov. 2025. Scalpel vs. Hammer: GRPO Amplifies Existing Capabilities, SFT Replaces Them. *arXiv preprint arXiv:2507.10616* (2025).
- [66] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.
- [67] Behzad Razavi. 2001. *Design of Analog CMOS Integrated Circuits*. McGraw-Hill.
- [68] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.
- [69] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [70] Rob A Rutenbar, Georges GE Gielen, and Jaijeet Roychowdhury. 2007. Hierarchical modeling, optimization, and synthesis for system-level analog and RF designs. *Proc. IEEE* 95, 3 (2007), 640–669.
- [71] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [72] Jinyi Shen, Zihao Chen, Ji Zhuang, Jiangli Huang, Fan Yang, Li Shang, Zhaori Bi, Changhao Yan, Dian Zhou, and Xuan Zeng. 2025. Atelier: An automated analog circuit design framework via multiple large language model-based agents. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [73] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. 2025. RL’s razor: Why online reinforcement learning forgets less. *arXiv preprint arXiv:2509.04259* (2025).
- [74] Yichen Shi, Zhuofu Tao, Yuhao Gao, Li Huang, Hongyang Wang, Zhiping Yu, Ting-Jung Lin, and Lei He. 2025. Amsnet 2.0: A large ams database with ai segmentation for net detection. In *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 242–248.
- [75] Yichen Shi, Zhuofu Tao, Yuhao Gao, Tianjia Zhou, Cheng Chang, Yaxin Wang, Bingyu Chen, Genhao Zhang, Alvin Liu, Zhiping Yu, et al. 2025. AMSnet-KG: A netlist dataset for LLM-based AMS circuit auto-design using knowledge graph RAG. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–37.
- [76] Yichen Shi, Ze Zhang, Hongyang Wang, Zhuofu Tao, Zhongyi Li, Bingyu Chen, Yaxin Wang, Zhiping Yu, Ting-Jung Lin, and Lei He. 2025. AMSBench: A Comprehensive Benchmark for Evaluating MLLM Capabilities in AMS Circuits. *arXiv preprint arXiv:2505.24138* (2025).

- [77] NS Karthik Somayaji and Peng Li. 2025. LLM-USO: Large Language Model-based Universal Sizing Optimizer. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [78] James A Svoboda and Richard C Dorf. 2013. *Introduction to electric circuits*. John Wiley & Sons.
- [79] Zhuofu Tao, Yichen Shi, Yiru Huo, Rui Ye, Zonghang Li, Li Huang, Chen Wu, Na Bai, Zhiping Yu, Ting-Jung Lin, et al. 2024. Amsnet: Netlist dataset for ams circuits. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5.
- [80] Qwen Team. 2025. QwQ-32B: Embracing the Power of Reinforcement Learning. <https://qwenlm.github.io/blog/qwq-32b/>
- [81] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [82] Prashanth Vijayaraghavan, Luyao Shi, Ehsan Degan, Vandana Mukherjee, and Xin Zhang. 2025. AUTOCIRCUIT-RL: Reinforcement Learning-Driven LLM for Automated Circuit Topology Generation. *arXiv preprint arXiv:2506.03122* (2025).
- [83] Deepak Vungarala, Sakila Alam, Arnob Ghosh, and Shaahin Angizi. 2024. Spicepilot: Navigating spice code generation and simulation with ai guidance. In *2024 IEEE International Conference on Rebooting Computing (ICRC)*. IEEE, 1–6.
- [84] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [85] Xinyue Wu, Fan Hu, Shaik Jani Babu, Yi Zhao, and Xinfei Guo. 2025. EasySize: Elastic Analog Circuit Sizing via LLM-Guided Heuristic Search. *arXiv preprint arXiv:2508.05113* (2025).
- [86] Yihong Wu, Liheng Ma, Lei Ding, Muzhi Li, Xinyu Wang, Kejia Chen, Zhan Su, Zhanguang Zhang, Chenyang Huang, Yingxue Zhang, et al. 2025. It takes two: Your grpo is secretly dpo. *arXiv preprint arXiv:2510.00977* (2025).
- [87] Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. 2025. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768* (2025).
- [88] Xingcheng Xu. 2025. The policy cliff: A theoretical analysis of reward-policy maps in large language models. *arXiv preprint arXiv:2507.20150* (2025).
- [89] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [90] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [91] Zitong Yang, Anon Zhang, Sam Wiseman, Xiang Kong, Ke Ye, and Dong Yin. [n. d.]. Memory retaining finetuning via distillation. ([n. d.]).
- [92] Yuxuan Yin, Yu Wang, Boxun Xu, and Peng Li. 2024. Ado-llm: Analog design bayesian optimization with in-context learning of large language models. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [93] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. 2025. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476* (2025).
- [94] Xi Yu, Dmitrii Torbunov, Soumyajit Mandal, and Yihui Ren. 2026. AutoSizer: Automatic Sizing of Analog and Mixed-Signal Circuits via Large Language Model (LLM) Agents. *arXiv preprint arXiv:2602.02849* (2026).
- [95] Haoyi Zhang, Shizhao Sun, Yibo Lin, Runsheng Wang, and Jiang Bian. 2025. Analogxpert: Automating analog topology synthesis by incorporating circuit design expertise into large language models. In *2025 International Symposium of Electronics Design Automation (ISED)*. IEEE, 772–777.
- [96] Wei Zhang, Qinggong Wang, Xiangtai Kong, Jiacheng Xiong, Shengkun Ni, Duanhua Cao, Buying Niu, Mingan Chen, Yameng Li, Runze Zhang, et al. 2024. Fine-tuning large language models for chemical text mining. *Chemical Science* 15, 27 (2024), 10600–10611.
- [97] Yifan Zhang, Yifeng Liu, Huizhuo Yuan, Yang Yuan, Quanquan Gu, and Andrew Chi-Chih Yao. 2025. On the design of kl-regularized policy gradient algorithms for llm reasoning. *arXiv preprint arXiv:2505.17508* (2025).
- [98] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2023. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems* 36 (2023), 55006–55021.
- [99] He Zhu, Junyou Su, Peng Lai, Ren Ma, Wenjia Zhang, Linyi Yang, and Guanhua Chen. 2025. Anchored Supervised Fine-Tuning. *arXiv preprint arXiv:2509.23753* (2025).